FINAL YEAR PROJECT INTERIM REPORT

683
Automatic Generation of Probing Questions

243UC247NS
YAM KAR YONG

Degree in Computer Science (Software Engineering)

February/2026

683
Automatic Generation of Probing Questions

BY

243UC247NS YAM KAR YONG

PROJECT INTERIM REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE

REQUIREMENT FOR THE DEGREE OF

Computer Science (Software Engineering)

in the

Faculty of Computing and Informatics

MULTIMEDIA UNIVERSITY
MALAYSIA

February/2026

Copyright

Declaration

I hereby declare that the work has been done by myself and no portion of the work contained in this report has been submitted in support of any application for any other degree or qualification of this or any other university or institution of learning.

Name of candidate: Yam Kar Yong

Faculty of Computing & Informatics

Multimedia University

Date: 11/11/2025

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my project supervisor, **Dr. Lim Tek Yong**, for their invaluable guidance, patience, and continuous encouragement throughout the duration of this Final Year Project. Their expertise and constructive feedback were instrumental in shaping the direction of this research, particularly in the complex areas of requirements engineering and Large Language Model integration.

I would also like to extend my sincere appreciation to the project moderator and the panel of examiners for their critical review and insightful suggestions. Their diverse perspectives helped refine the system architecture and ensured the academic rigor of this "Hybrid Intelligent Agent" design.

I am also grateful to the **Faculty of Computing and Informatics (FCI)** at **Multimedia University** for providing the necessary infrastructure and learning environment that made this project possible. The knowledge gained during my studies here has been the foundation upon which this system was built.

Special thanks go to the industry professionals and peers who participated in my data collection survey. Their honest feedback on the challenges of manual requirements elicitation provided the crucial data needed to justify the development of the *AI Probing Question Generator*.

Finally, I must express my profound gratitude to my parents and family for their unwavering support and understanding during the many late nights spent on development and documentation. Their belief in my potential has been my greatest motivation. To my friends and course mates, thank you for the camaraderie, the shared knowledge, and the moral support that kept me going until the completion of this report.

Abstract

Requirements elicitation is a critical stage in software development, often suffering from incomplete or unclear stakeholder input. Many software failures occur because analysts fail to ask the right probing questions or misunderstand stakeholder needs. To overcome this challenge, this project proposes an intelligent system named the "AI Probing Question Generator," which focuses on the automatic generation of probing questions to refine and clarify software requirements. The proposed system will be implemented using Natural Language Processing (NLP) and Large Language Model (LLM) techniques to analyze stakeholder descriptions, detect ambiguities or gaps, and produce context-relevant probing questions. It is designed to iteratively generate follow-up questions based on responses, suggest refinements for better elicitation, and validate requirements through targeted inquiries to ensure completeness and accuracy. By integrating these capabilities, the system aims to reduce human error, improve communication between stakeholders and engineers, and increase the clarity of software requirements, ultimately leading to enhanced software quality and development success.

Table of Contents

List of Tables

List of Figures

List of Abbreviations/Symbols

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CA	Conversational Agent
CSCW	Computer-Supported Cooperative Work
FCI	Faculty of Computing and Informatics
FYP	Final Year Project
HCI	Human-Computer Interaction
LLM	Large Language Model
MMU	Multimedia University
MVC	Model-View-Controller
NLP	Natural Language Processing
PCI-DSS	Payment Card Industry Data Security Standard

Abbreviation	Meaning
QA	Quality Assurance
RE	Requirements Engineering
SDLC	Software Development Life Cycle
UAT	User Acceptance Testing
UML	Unified Modeling Language

List of Appendices

Chapter 1: Introduction

1.1 Overview

In software development, the requirements engineering phase is often the most influential in terms of the ultimate success or failure of the system. This phase involves technical inquiries, analytical processes, and validation of requirements of the stakeholders, in order to construct a system that meets their needs. A primary issue with the requirements engineering phase is that stakeholders tend to utter requirements in vague or incomplete terms. This lack of clarity on the part of the stakeholders, essentially makes requirements engineering a guessing game. Because of ambiguity, the project can encounter delays, and as a result the cost to develop the project can increase, not to mention the likelihood that project will need to be reworked significantly.

The inspiration for the AI Probing Question Generator came from the fact that requirements engineers tend to do manual interviews to extract information from the stakeholders. Many engineers rely on their own subjective experiences or predetermined questions, thereby omitting critical information, especially novice engineers. The AI Probing Question Generator is designed to enhance and automate the engineering interviewing process, using the most current LLM and NLP technologies. Thus, the AI Probing Question Generator aims to build a system that will help engineers formulate questions to better understand what the stakeholders need and to help them streamline the requirements.

1.2 Problem Statement

The main problem with AI Probing Question Generator is the manual requirement elicitation process of eliciting, which is inefficient and highly inconsistent. When people discuss with the stakeholders what they want from the system, the explanation is very informal and the engineers have to go into many follow up questions to explain, clarify, and confirm different details. Without the right guide questions, critical information is often left behind, and therefore the requirements are either incomplete or in conflict. The quality of the end product greatly depends on how well requirements are gathered in the early beginning, and thus, a need exists for an intelligent system that will automatically generate the right probing questions to help in the gathering of all the needed clear requirements prior to the start of any software development.

1.3 Project Objectives

The goals of this project are to examine the prior works on the automated generation of probing questions, to create an NLP-based system that automatically generates probing questions based on input from the stakeholders, and to analyze the system concerning estimating the accuracy, relevance, and the overall contribution to the completeness and clarity of the requirements.

1.4 Project Scope

The AI Probing Question Generator project aims to create a web-based application to automate the support of requirements-elicitation processes. The system receives a text input from the stakeholders that outlines what they would like the software to accomplish. The system then generates specific questions to reveal the details that are missing and iteratively generates follow-up questions based on the stakeholder's reply to the previous ones so that the system can elicit the remaining requirements.

1.5 Project Limitations

The system has several limitations. The first limit is that this system will focus on text-based interviews and the AI Probing Question Generator will rely on NLP and a few predefined templates synthesized by LLM. The system is designed to be evaluated based on a small-scale assessment using pre-defined stakeholder scenarios and not on real industry case studies.

1.6 Methodology

In this project, the AI Probing Question Generator is being developed using the Waterfall software development life cycle (SDLC) model. The choice of model is based on the linearity of the Waterfall model; each phase of development is completed in series (one after another), which is ideal for this model as the project has specific goals, a well-defined scope, stable requirements, and focuses on the automation of the elicitation of requirements. The Waterfall model facilitates the completion of each phase in a sequential manner and requires extensive documentation to support each milestone. As such, the Waterfall model is ideal for use in the academic environment as the project requirements are defined in in detail and are unlikely to change throughout development.

Requirements Gathering and Analysis

During this phase, I focused on documenting the requirements of the project in correlation to the goals and constraints defined for the project. The needs of the stakeholders were examined as to the need for textual input to the system from the requirements engineer(s) and business analyst(s), the need for the system to generate probing questions that are specific to the relevant context, the need for the system to include functionality for iterative follow-up questions, and the need for the system not to include redundant follow-up questions. As this was necessary To identify the balance of the functional and non-functional requirements, the system was to focus on the use of an NLP LLM-enhanced framework for the purposes of text-based interviews. I was able to identify and document a comprehensive set of project requirements.

System Design

In parallel with finalizing requirements, the system architecture was designed. This was composed of high-level design (overall structure of the web application, user interface design to capture input and display the questions) and low-level design (prompt design of the LLMs, question category templates, design of conversation history to avoid redundancy, and incorporation of various NLP

tools). Use case, data flow, and sequence diagrams, among others, were designed to capture the iterative question generation process to dig deeper during the elicitation process.

Implementation

In this phase, the design was coded. To this end, a web-based application was designed around the chosen technology (i.e., a frontend framework for interface, a backend for stakeholder input processing and LLM model API calls). Core functionalities were developed to include submission of unstructured text descriptions, auto-creation of primary probing questions from a combined NLP and LLM engine, and generation of iterative follow-up questions based on a conversational context and state.

Verification and Testing

After the completion of the implementation phase, the system was evaluated extensively. This encompassed the unit testing of components (ex. the system's ability to generate questions accurately), the iterative flow and redundancy testing, and system testing using sample data. Given the system's purpose, usability was evaluated to determine whether the questions were produced in a clear, relevant and context-specific manner.

Evaluation and Maintenance

The last phase focused on a limited evaluation employing fictitious stakeholder scenarios to measure the performance of the system on questions regarding the accuracy, relevance, and completeness, and the clarity concerning the requirements. Human evaluation (e.g. rating by experts) and benchmarking (e.g. evaluation against manual questioning) were used. Some future maintenance issues (e.g. prompt adjustments and refinements, improvements in the system's ability to handle increased volume of work or expansion) have been noted.

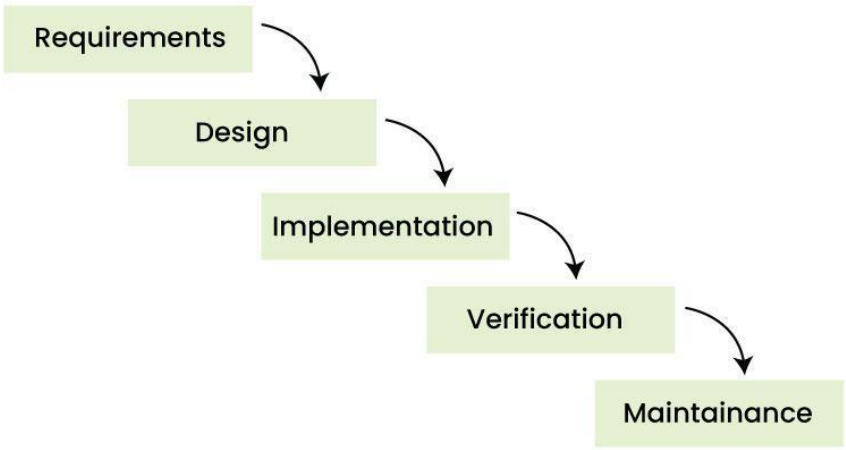


Figure 1.1 Waterfall Model diagram

1.7 Target Audience

The audience for this system includes individuals and groups in the software development domain who are involved in collection, analysis, or comprehension of system requirements. This also includes business engineering analysts and interviewers, who are the main users of the system, as the AI Probing Question Generator helps them in formulating probing questions to avoid omission of requirements and to enhance precision in elicitation. The AI Probing Question Generator can also be used by project managers and team leaders to facilitate elicitation of requirements and monitor the extent to which requirements have been addressed. The system can also be utilized by the novice and student community of software engineering for the purposes of learning to ask the right questions and perform the right activities in the process of requirement elicitation. The AI Probing Question Generator can also be used by small-scale software firms, or start-up companies with little manpower and inexperienced analysts, to systematize the process of requirement elicitation and minimize the risks associated with vague and incompletely defined requirements.

1.8 Summary

The aim of the project is to create a system, Automatic Probing Question, able to formulate follow-up questions based on a user's input (e.g., text, responses, or statements). Manual question crafting negatively affects consistency, time, and domain knowledge. Probing questions are important in guiding learning and interaction within a conversation system.

The project employs Natural Language Processing (NLP), and Large Language Models (LLMs) or other machine-learning-based models to determine and formulate relevant context probing questions. The project consists of data collection, model crafting, prototype creation, and subsequent assessment. The outcome is a tool meant to assist/interact with the Requirement Engineer, interviewer, or user of a LLM learning model. Other anticipated outcomes comprise smarter critical support, higher engagement, and question generation.

Chapter 2: Literature Review

2.1 Overview

The ability to develop conversational agents (CAs) and large language models (LLMs) can assist in automating the information and requirements elicitation processes, especially in semi structured interviews. While traditional human interviews can be resource intensive, suffer from cognitive interviewer bias, and cognitive load, they are still the primary means to obtain rich, qualitative data in human-computer interaction (HCI), computer-supported cooperative work (CSCW), and software requirements engineering (RE). Recent studies suggest the automate elicitation process through CAs and LLMs by means of generating relevant follow-up questions, carrying out full interviews, and/or assisting interviewers in real-time. This review focuses on three studies: a landmark work on general CA design from 2024 and 2025 preprints on CAs and LLMs in software requirements elicitation. These studies illustrate the evolution from simple CA design to sophisticated LLM implementation.

2.2 Conversational Agents for Follow-up Questions in Semi-Structured Interviews

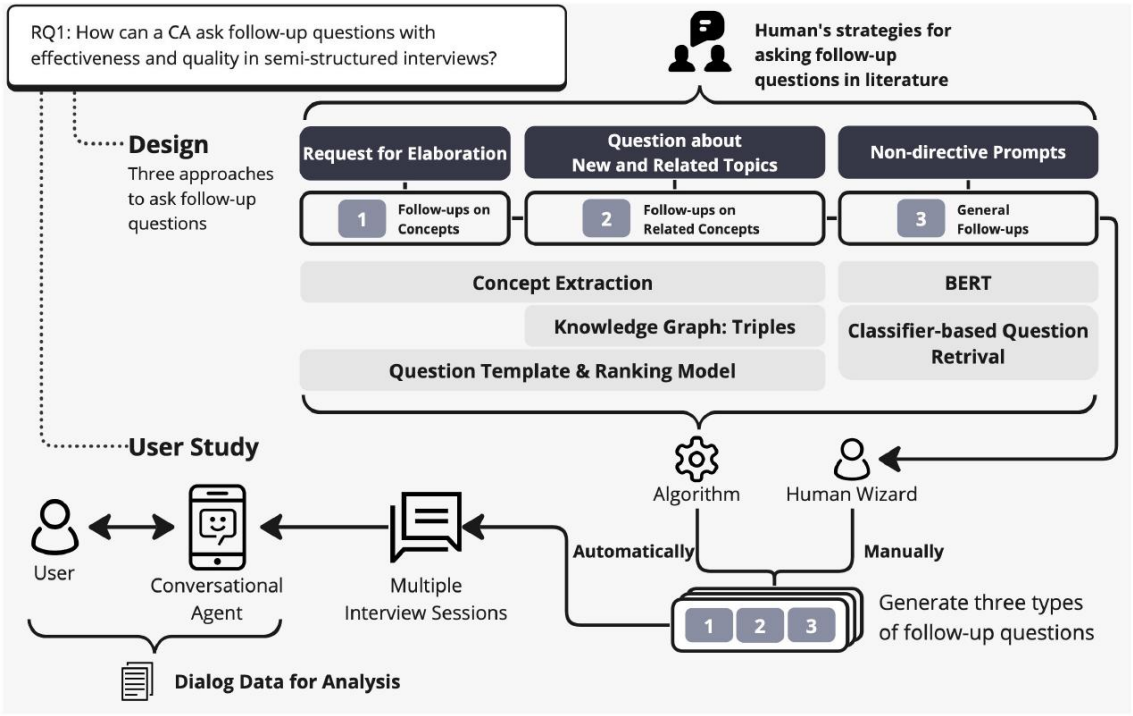
An initial systematic study on conversational agents (CAs) being able to ask follow-up questions for information elicitation in semi-structured interviews is provided by Hu et al. (2024).

2.2.1 Design and Implementation

The authors suggest three types of follow-up questions based on human interviewing techniques: (1) follow-ups on concept (elaboration on a particular entity mentioned by the user), (2) follow-ups on related concepts (broadening the discussion using knowledge graphs such as ConceptNet), and (3) general follow-ups (vague prompts such as “Why?” or “Can you say more?”). The authors’ approach combines rule-based concept extraction and population of ontologies and questions with templates and pre-trained language models, which only uses a small dataset (2,214 question-response pairs created from 11 human-human interviews about smart device usage).

2.2.2 Evaluation and Findings

A within-subject user study (N=26) was conducted to compare follow-ups that were generated by algorithms and those that were created by humans. The study showed that all three types (both related concept and concept based) had a positive relevance and lower drop-out rates, while general follow-ups had a positive relevance and facilitated a more informative response from the user. Although the algorithmic questions had the same level of human generated questions in regard to informativeness, they were likely to have a higher level of drop-out rates. Human generated questions showed the most nuanced and skilled level of phraseology as evidenced in qualitative analysis.

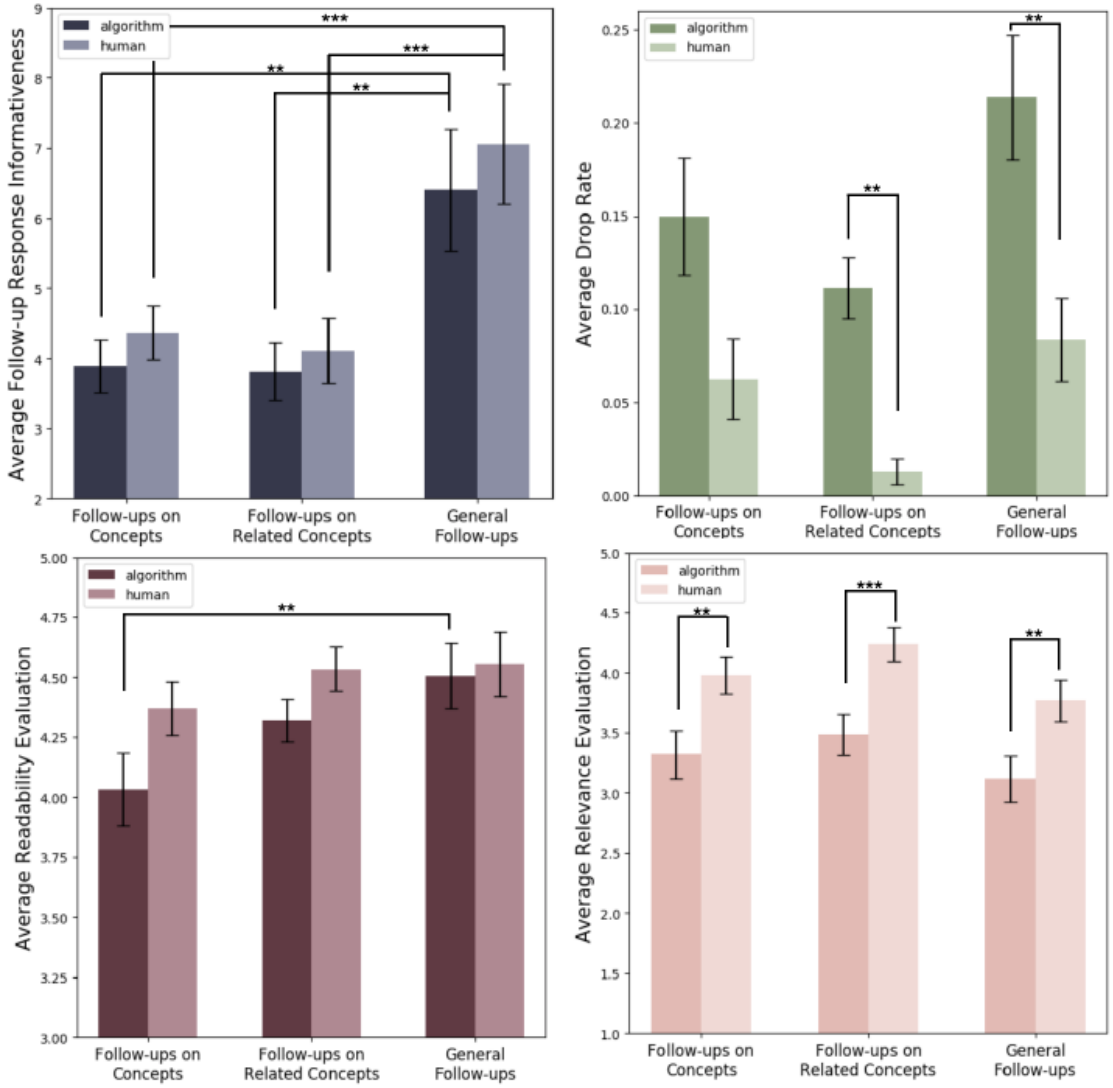


2.1 Figure Three Type of follow up question inspired by human behavior

Table 1. Three types of follow-up questions.

Follow-up Question	Synopsis	Example Dialog
	The concept *smart speaker* is mentioned	Q: Do you use any smart devices? A: Yeah. I have a *smart speaker*.
Follow-ups on Concepts	Requests for specification or definition of a mentioned concept	Q: What kind of *smart speaker*?
Follow-ups on Related Concepts	Raising a new concept related to a mentioned concept	Q: Would you use the *smart speaker* to *set alarms*?
General Follow-ups	Questions or prompts without mentioning any concept	Q: Anything else?

2.2 Three types of follow up questions example table



2.3 Figure show Comparison of 3 types of follow up question

2.3 Automating Full Requirements Elicitation Interviews with LLMs

Korn et al. (2025) incorporate LLMs into the automation paradigm for the first time by developing a GPT-4o powered chatbot that performs fully autonomous requirements elicitation interviews called LLMREI.

2.3.1 Approach

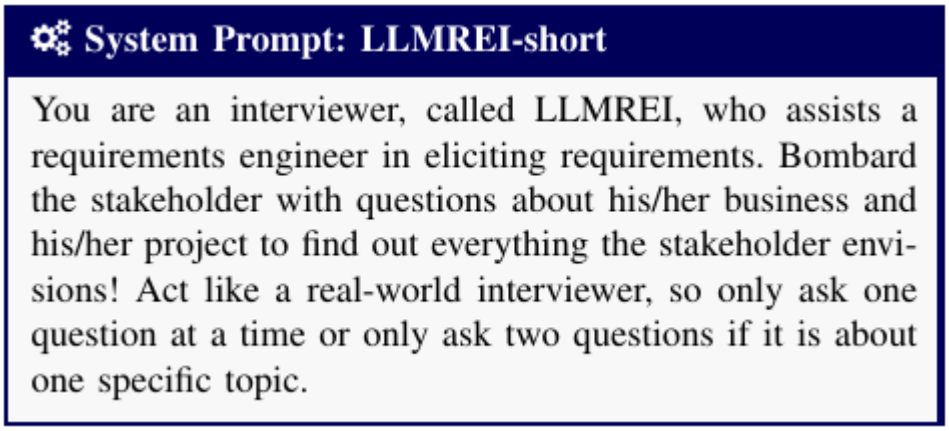
The LLMREI system uses the following prompting methods that include, but are not limited to, zero-shot (LLMREI-short) where the LLM is instructed to create a follow-up question, and least to most (LLMREI-long) where the LLM is given several instructions to complete a set of subtasks) to direct the LLM to initiate interviews, create follow-up questions that are adaptive, avoid common interviewer mistakes (such as asking leading questions), and summarize requirements. Considerable effort was given to fine-tuning but it was ultimately decided that the performance was not satisfactory.

2.3.2 Evaluation and Results

Of the 33 interviews that were simulated, LLMREI produced a competitive error rate to humans, obtained an adequate share of relevant requirements (as high as an average of 70% out of all relevant requirements in optimal settings), and provided questions that were contextually rich and relevant. The system performed remarkably in the domains of scalability and consistency lending support for LLMREI to be used in large volumes of RE projects.

2.3.3 Positioning Relative to Prior Work

This work builds upon the basic designs of CA, such as that of Hu et al. (2022), and uniquely demonstrates that contemporary LLMs are capable of conducting complete interviews in a fully professional and structured field, thus moving the focus from hybrid rule-based systems to fully prompting-based systems.



2.4 Figure of Zero-shot Prompting (LLMREI-short)

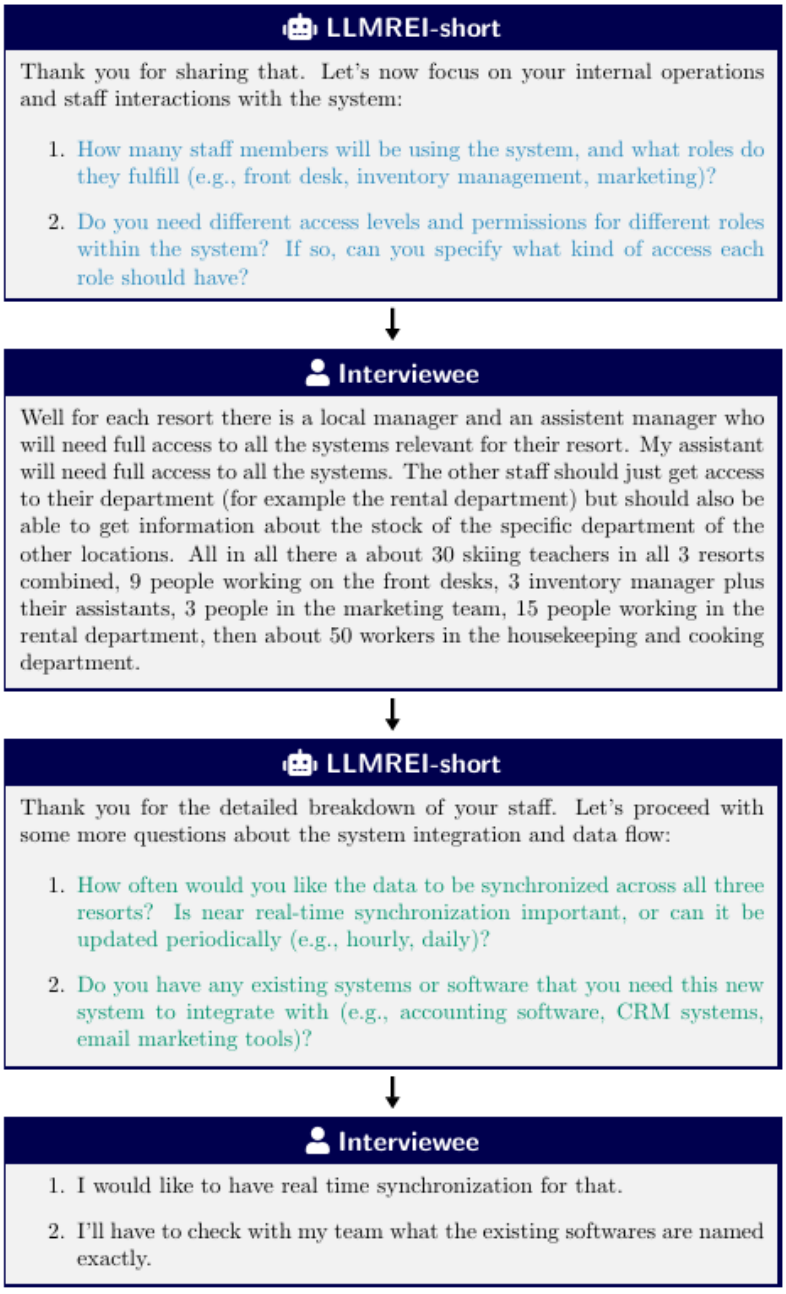


Fig. 1. Conversation excerpt from an interview conducted by the LLMREI-short bot, highlighting its ability to transition smoothly between probing (blue) and follow-up (green) questions.

2.5 Figure of prompting workflow (zero-shot)

System Prompt: LLMREI-long

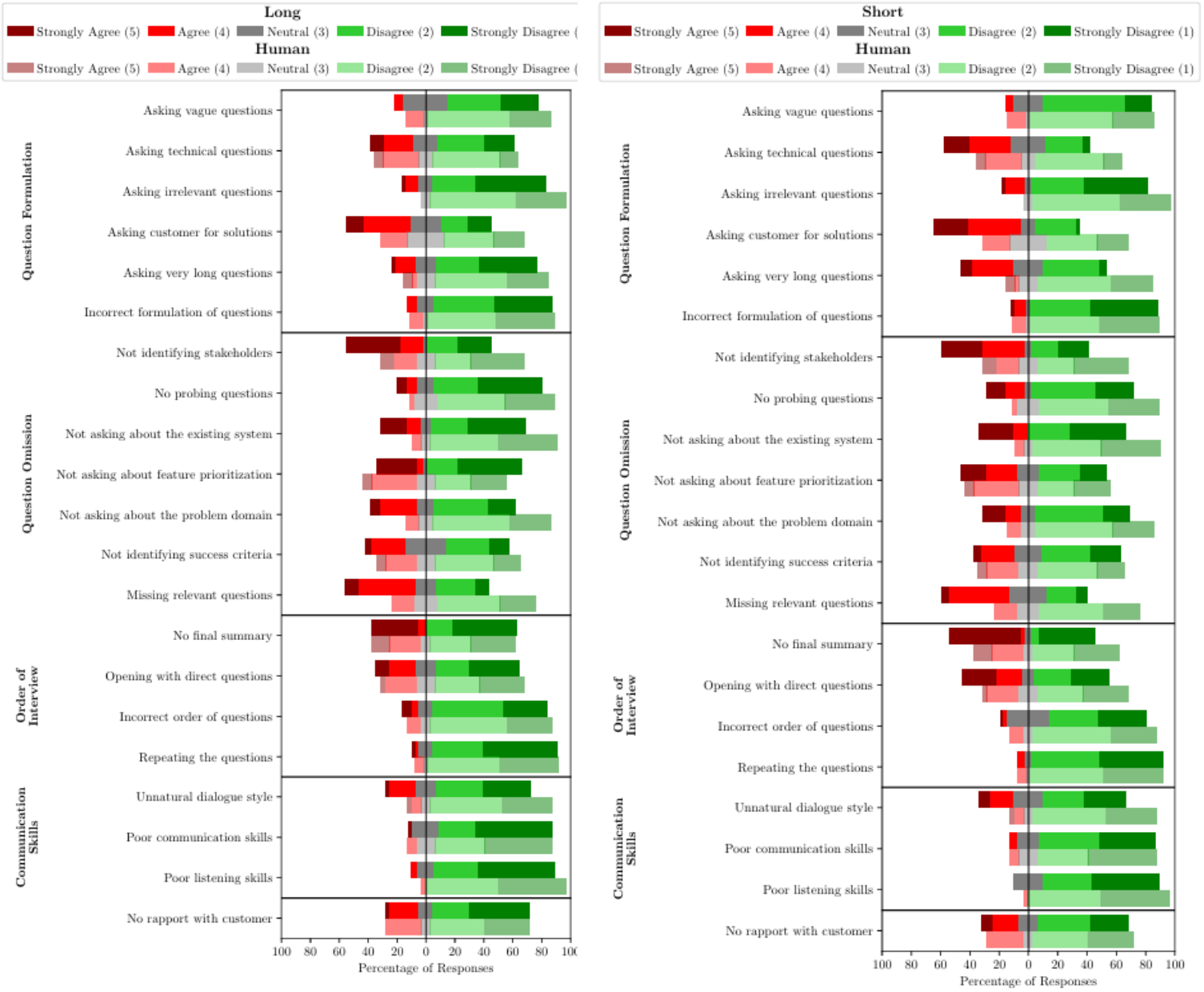
You are an interviewer, called LLMREI, who assists a requirements engineer in eliciting requirements.

Goal: The purpose of this chat bot is to conduct comprehensive and effective requirements elicitation interviews with stakeholders, ensuring all necessary information is gathered to support project development.

{Interview Cookbook}

Maintain professionalism throughout the interview. Adjust questions based on the stakeholder's role, education level, and domain knowledge. Adapt questioning style to fit the flow of the stakeholder's responses. Actively listen to differentiate between stated needs and actual needs. Let the customer create scenarios. Example: "Please visualize the first page of your application and explain how you would interact with it step-by-step."

2.6 Figure of Least to most prompting technique (LLMREI-long)



2.7 Result of Percentage of Responses compared with zero-shot(short) and least to most (long) techniques

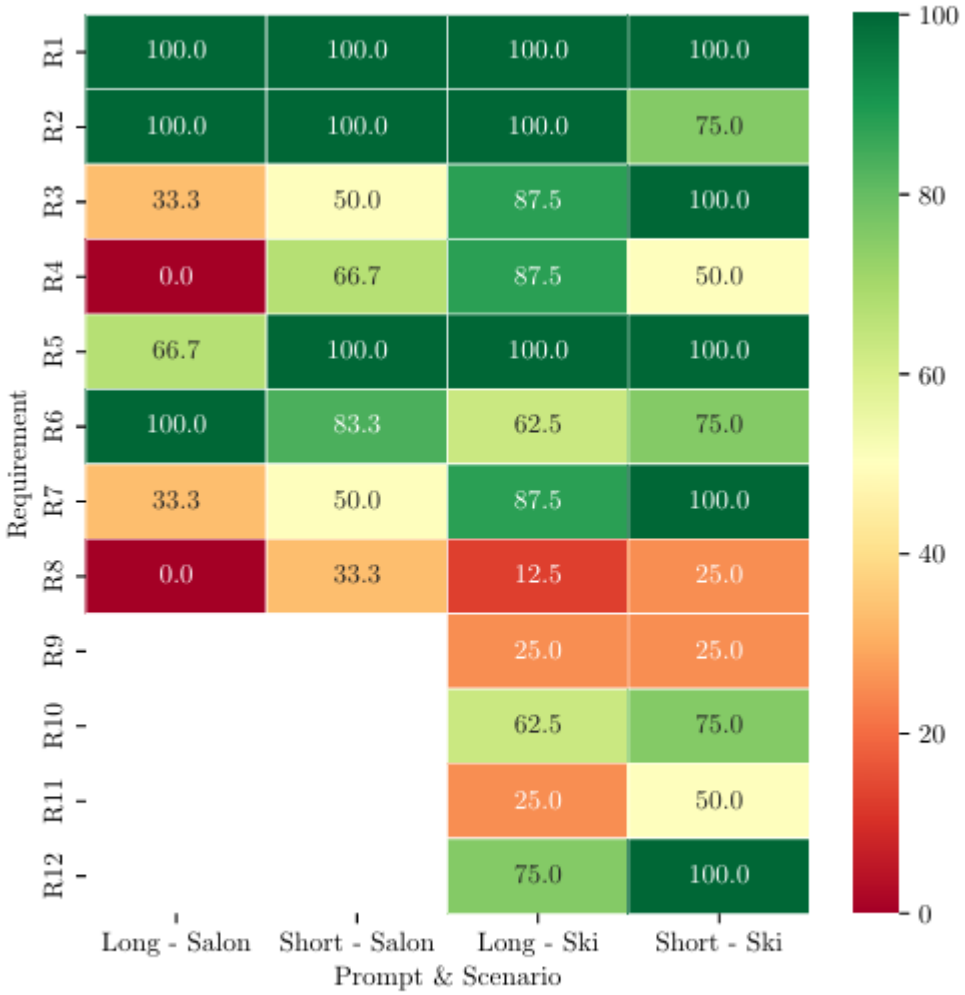


Fig. 5. Percentage of interviews in which a specific requirement was elicited.

2.8 Figure of Accuracy of Percentage of Two techniques compared with 2 humans through requirement elicitation

2.4 LLM-Assisted Follow-up Question Generation in Requirements Elicitation

Narrowing down the focus to real-time generation of follow-up questions during requirements interviews, Shen et al. (2025) employs GPT-4o to supplement human interviewers.

2.4.1 Framework and Methods

The authors construct a taxonomy of 14 types of common mistakes by interviewers (e.g., questions are too vague, failure to probe deeper into unexamined assumptions) from the RE literature. Two prompting types are contrasted: minimally guided (lead by a single context) and mistake-guided (instructed to avoid certain mistakes). The 14 interviews were conducted to build the directory service application and were used for the experiments.

2.4.2 Key Finding

Most effective follow-ups require minimal prior context (0–1 turns). Minimally guided LLM questions matched human performance on clarity, relevance, and informativeness. Mistake-guided prompting significantly outperformed humans (68% preference, $p < 0.05$) by systematically avoiding errors.

2.4.3 Implications

This work highlights the value of structured guidance in prompting and suggests LLMs are particularly strong as real-time assistants rather than fully autonomous agents in high-stakes RE contexts.

```
You are an AI agent capable of generating context
summaries. During a requirements elicitation interview
with an interviewee about how the interviewee conducts
{interview domain}, the INTERVIEWEE and INTERVIEWER
have had the following conversation: {interview turns}.
Generate a follow-up question that the INTERVIEWER
should ask next based on the conversation. Restrict
your response to only show the follow-up question without
explanation.
```

(a) Prompt 1 to Generate Minimally Guided Questions

```
You are an AI agent capable of conducting requirements
elicitation interviews. During a requirements elicitation
interview with an interviewee about how the interviewee
conducts {domain keyword}, the INTERVIEWEE said
'{interviewee speech}'. Then the INTERVIEWER asked a
follow up question by saying '{interviewer question}'.
Standard: {mistake criterion}. Please classify based
solely on whether the INTERVIEWER's response meets
this specific standard, and refrain from using any
other standards related to follow up questions when
you classify. If the INTERVIEWER's response meets this
standard, output 'Yes', otherwise output 'No'. Restrict
your response to output only 'Yes' or 'No' without
explanations.
```

(b) Prompt 2 to Classify Follow-up Questions

```
You are an AI agent capable of conducting requirements
elicitation interviews. During a requirements elicitation
interview with an interviewee about how the interviewee
conducts {domain keyword}, the INTERVIEWEE said
'{interviewee speech}'. Generate a follow-up question
that meets the following criterion based ONLY on what the
INTERVIEWEE said, and restrict your response to only show
the follow-up question without explanation. Criterion:
{mistake criterion}
```

(c) Prompt 3 to Generate Mistake-Guided Questions

2.9 Prompt strategies from Shen (2025) findings

(b) Classification Results For Each Mistake Type

Mistake Type	Human	GPT-4o	Agreement Rate	Avoidance Rate
Fail to elicit tacit assumptions	19	25	66.7%	76.7%
Fail to consider alternatives	30	28	93.3%	26.7%
No clarification when unclear	20	16	60.0%	70.0%
No clarification when contradictory	7	2	83.3%	83.3%
Fail to elicit tacit knowledge	21	20	70.0%	90.0%
Ask a generic, domain-independent question	7	10	90.0%	96.7%
Ask a question that is too long or articulated	11	7	86.7%	90.0%
Use jargon	7	1	80.0%	100.0%
Ask a technical question	5	1	86.7%	100.0%
Ask a question inappropriate to user's profile	5	0	83.3%	100.0%
Ask for solutions	6	2	86.7%	100.0%
Ask a questions that involves multiple kinds of requirements	7	12	76.7%	60.0%
Ask a vague question that leads to multiple interpretations	27	27	93.3%	93.3%
Ask a vague question which could infer no reasonable meaning	5	12	76.7%	93.3%
Total	177	163	81.0%	84.3%

2.10 Classification Results for each mistake type compared Human vs GPT

(c) Relevancy, Clarity and Informativeness Results

Metric	Relevancy	Clarity	Informativeness
Human Avg Score	3.5	3.9	3.6
GPT-4o Avg Score	4.4	4.5	4.1
Human Win Rate	21.1%	17.2%	25.8%
GPT-4o Win Rate	59.4%	42.2%	47.7%
Tie Rate	19.5%	40.6%	26.6%
p-value	1.21×10^{-10}	1.34×10^{-6}	1.77×10^{-5}

2.11 GPT vs Human Relevancy, Clarity and Informativeness

2.5 Trends and Gaps Across the studies

The three papers trace the evolution of mixed rule-based/MLP designs that take small data sets (Hu et al., 2024) to prompting-based LLM systems that require little data (Korn et al. and Shen et al., 2025). General semi-structured interviews tend to benefit from typing strategies for follow-up questions. RE-specific work highlights error avoidance and domain constraints. Themes that prevail include the high informativeness of the LLM/CA questions, human-level and sometimes superhuman performance in controlled environments, and the remaining difficulties associated with nuance and the drop rate. Gaps in the literature still exist for hybrid human-LLM workflows, long-term engagement, and domain adaptation beyond smart devices and directory services, and in real (as opposed to simulated) environments

2.6 Comparison Each Techniques

Table 2.1: Techniques Comparison Tables

Feature	**Paper 1: Mistake-Driven Prompting** **(Shen.)**	**Paper 2: Strategy-Based Generation** **(Hu et al.)**	**Paper 3: Least-to-Most Prompting (LLMREI)** **(Korn)**
Core Technique	Uses a **Taxonomy of Mistakes** (e.g., ambiguity, compound questions) to prompt the LLM on what *not* to do.	Uses **3 Explicit Strategies**: (1) Concept-based, (2) Related Concept-based, and (3) General follow-ups.	Uses **Least-to-Most Prompting** to break the interview into smaller sub-tasks (Context → Question).
Key Strength	**High Precision.** Producing "clean" questions is easier when you explicitly ban bad patterns.	**High Context Awareness.** Ensures questions aren't just random; they follow a logical "investigative" path.	**Process Oriented.** Handles the flow of a full interview better than single-question generators.
Limitation	**Static Rules.** It relies on a fixed list of "mistakes." It might miss new types of errors not in its list.	**Rigid Structure.** It forces questions into 3 buckets, which might feel robotic or repetitive after a while.	**Generic Knowledge.** Without fine-tuning (which failed in the paper), it lacks specific domain knowledge.

2.7 Summary

The research for automating requirements elicitation shows that a shift has occurred from rigid frameworks to adaptable LLM systems. This is evident in the review conducted for the three main paradigms of Strategy-Based Generation, Least-to-Most Prompting, and Mistake-Driven Prompting. While Hu et al. (2024) enhanced the structural variety of LLMs with specific questioning strategies, Korn et al. (2025) showed that “Least-to-Most” is more suited to handling the context and flow of an interview, and Shen et al. (2025) reached the highest level of precision by explicit prompting against typical interviewer errors. This project addresses the shortcomings of the individual approaches by proposing a Hybrid Intelligent Agent that integrates the three approaches into a single pipeline. The “AI Probing Question Generator” will implement context management from Korn et al. to understand an agent’s intent, selection of strategy type from Hu et al. to identify a relevant question type, and mistake validation from Shen et al. to filter out errors. This will ensure that the generated questions are context-coherent, structurally diverse, and methodologically robust.

Chapter 3: Requirements Analysis

3.1 Overview

The first and most important step of any project is to define the requirements of stakeholders. For the system entitled AI Probing Question Generator, they first tried to understand the difficulties involved in the manual process of requirements elicitation. They specifically tried to understand the difficulty analysts experience in developing contextually relevant and effective probing questions when stakeholders provide descriptions that are vague/ incomplete. Here, the goal was to articulate a precise, minimalistic, and functional prototype that is capable of yielding real-time probing questions of adequate quality. In order to elaborate the requirements, fact finding methods were used to collect the insights of the users (business analysts, product owners, and software engineers) which also helped to validate the problem and confirm the need for a prototype/ minimalistic assistant.

3.2 Fact-Finding Techniques

3.2.1 Justification

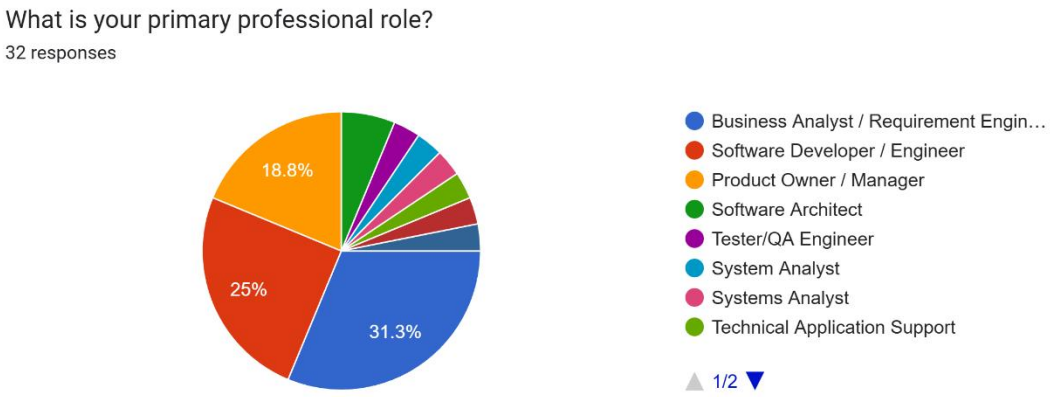
Among the multiple fact-finding methods identified in the preliminary study, the questionnaire stood out because it is time efficient, cost-effective, and allows participants to remain anonymous. Ease and opportunity to remain anonymous encourages participants to provide meaningful feedback, especially for questions that may be perceived as sensitive and biased, such as, inefficiencies in elicitation. In this case, the questionnaire helped in collecting both the qualitative and the quantitative data needed in identifying and/or describing the potential challenges of elicitation to more participants than the qualitative-intensive interview approach or participatory observation methods. The relative quantitative and qualitative data from the questionnaire was augmented by pertinent literature review. The questionnaire was more appropriate for identifying and describing the challenges of elicitation than any other method because beyond measurement, it was evidence-based, cost-effective, and was able to be distributed and collected online.

3.2.2 Questionnaire

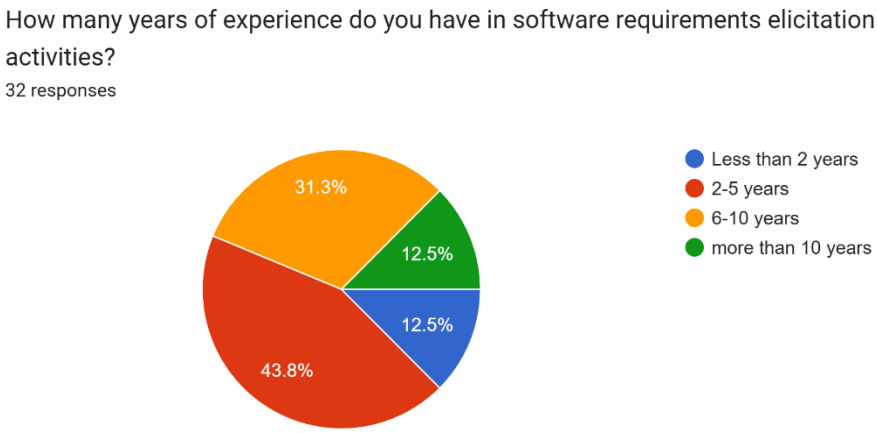
The questionnaire, completed using Google Forms, was administered to thirty (30) participants sourced from professional networks, for example, LinkedIn groups dedicated to business analysis or software engineering. The questionnaire includes both closed (multiple choice, Likert scale) and open questions to ensure quantitative data is collected, as well as to provide space for qualitative data. The topics of the questions include, but are not limited to, demographics, challenges of elicitation, the necessity of probing questions, and the features of the tools designed to address the challenges. The time to complete was purposefully designed to take between 5 and 10 minutes in order to encourage higher response rates, otherwise the response rate would be negatively affected.

3.2.3 Analysis on Results

The survey garnered 30 responses from various roles, the majority being Business Analyst/Requirements Engineer, followed by Product Owner/Manager, Software Developer/Engineer and others, including Technical Application Support, Software Architect, QA Engineer. Their level of experience ranged from less than 2 years, 2-5 years, 6-10 years, to more than 10 years.

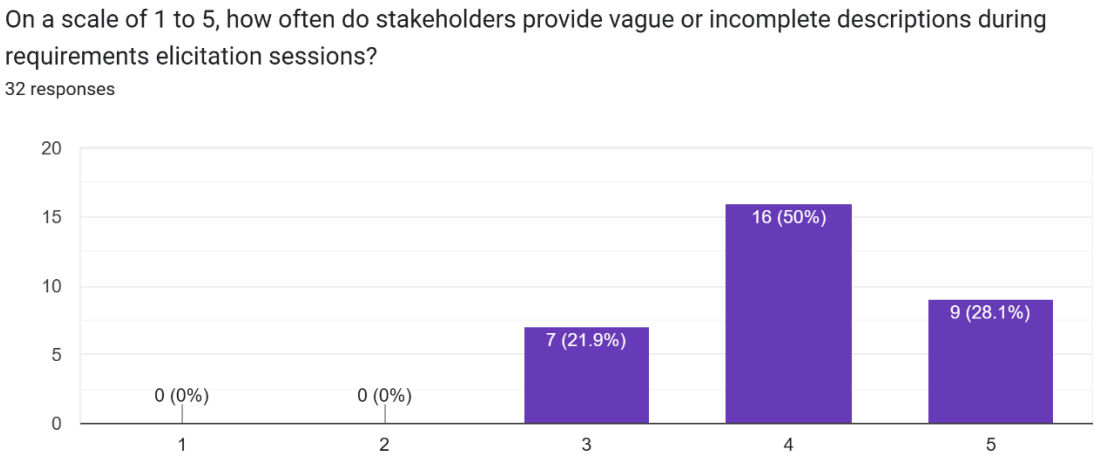


3.1 Figure of Percentage on Primary Professional role



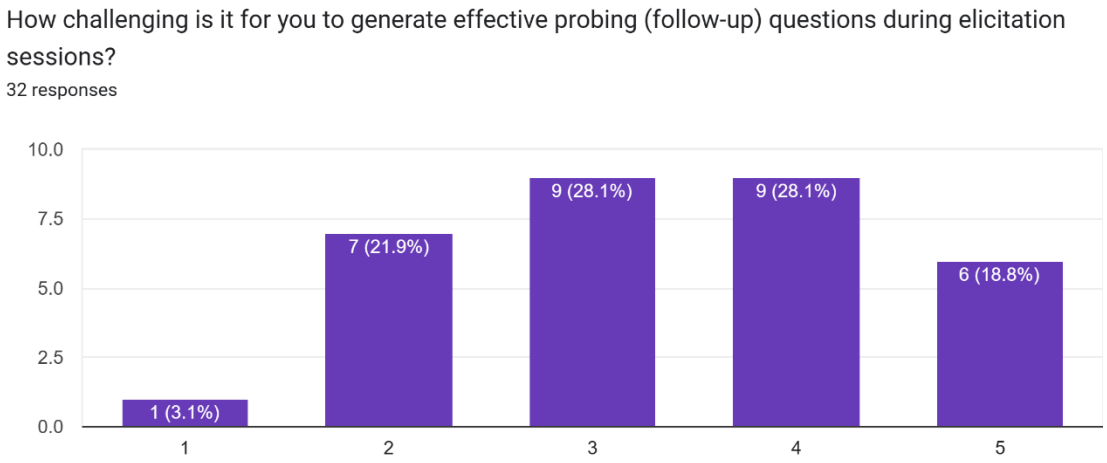
3.2 Figure of Percentage on year experience in software requirement elicitation

Quantitative results have validated the primary issue. On a 1 to 5 scale, the frequency of vague/incomplete explanations given by the stakeholders received an average score of 4.2, with the majority of the responses being 4 and 5.



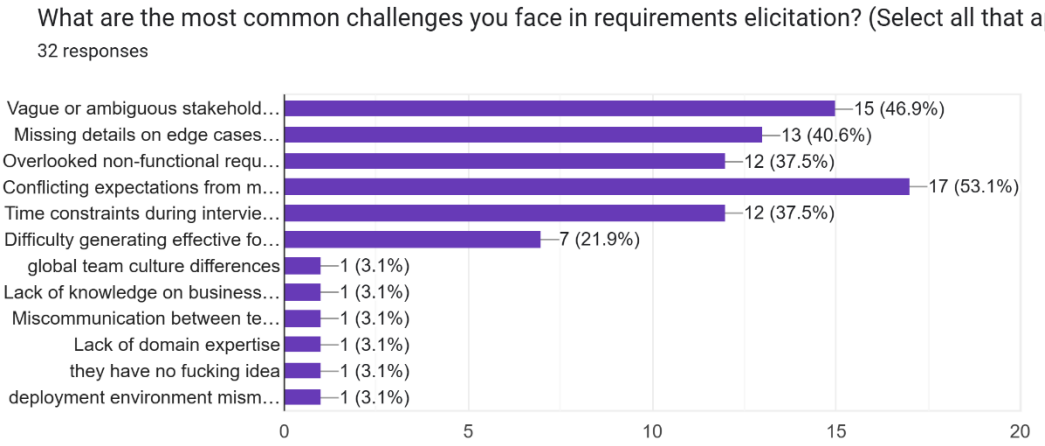
3.3 Figure of scaling 1-5 on how often stakeholder provide incomplete description

The average level of difficulty posed by the stakeholders in generating effective probing questions was rated 3.5-4, which is considered to be moderate to a large amount of difficulty.



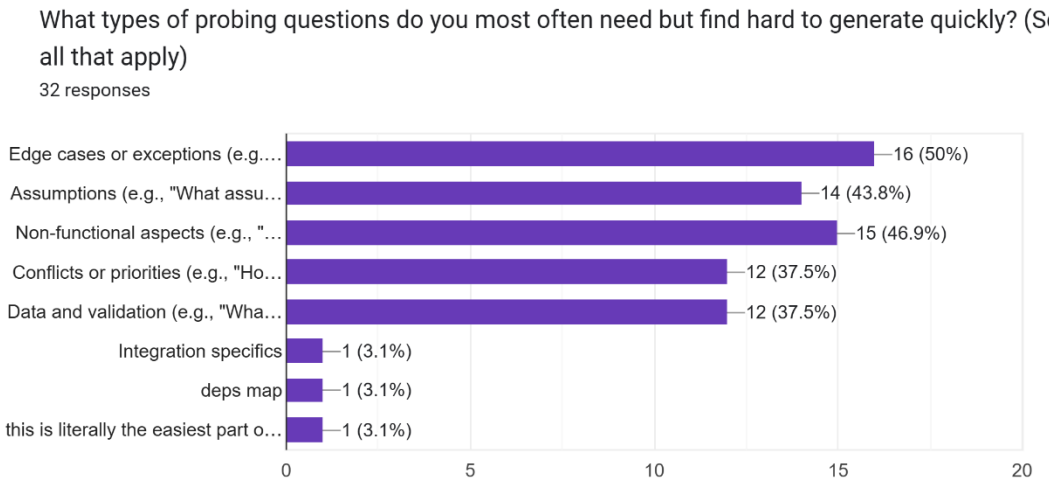
3.4 Figure of scaling 1-5 on challenge level to generate effective probing question

Common challenges cited by the majority include vague language, missing edge cases, overlooked non-functional requirements, conflicting expectations, and time constraints.



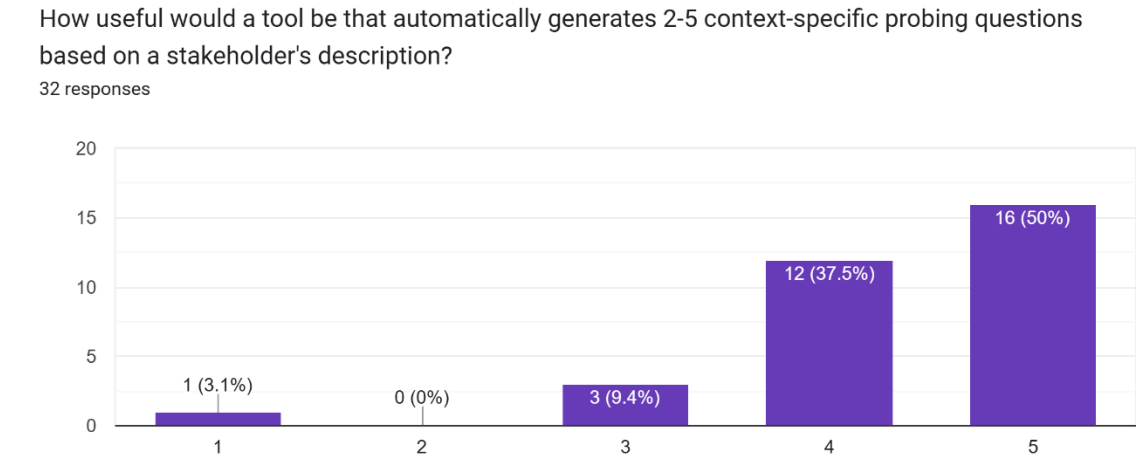
3.5 Figure of Result on most common challenge face in requirement elicitation

Respondents often required probing questions about edge cases or exceptions, the non-functional aspects of the system such as performance and security, assumptions, data validation, and conflict/priority issues.



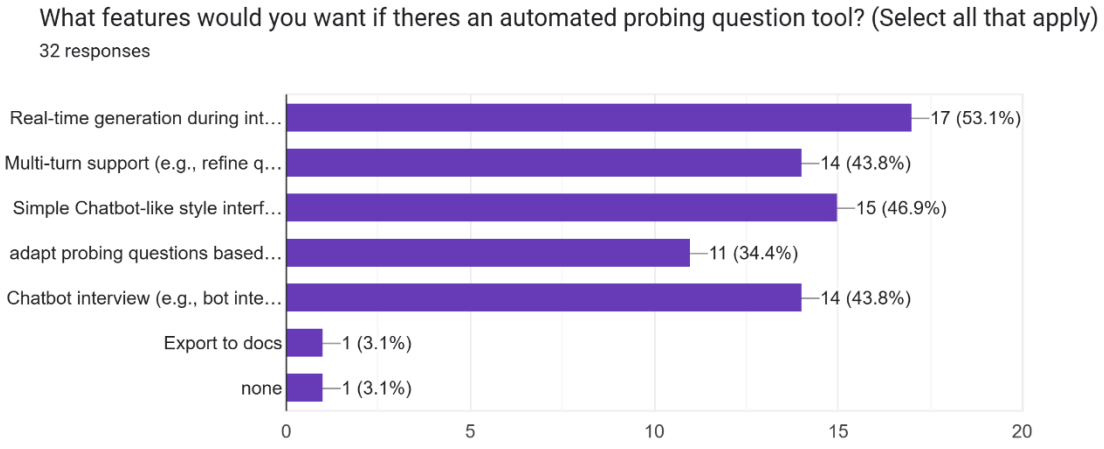
3.6 Figure of Result in which probing questions most often need but hard to generate quick

The usefulness of an automated tool generating context-specific probing questions averaged 4.4 out of 5, with most rating it 4 or 5.



3.7 Figure of 1-5 Result on how useful if auto generate probing question

Real-time generation during interviews, multi-turn capability, simplistic chatbot-like interfaces, adaptability to different roles of stakeholders, and stakeholder interviews in a chatbot format were the desired features.



3.8 Figure of Responses on features in probing question tool

Poor probing rework comments included missed security requirements, payment edge case and priority conflicts, and suggestions on security, domain specific on premise hosting, meeting tool integration, and project history learning. The results confirmed the automation need for probing assistance, shaping its focus.

3.3 Requirement

3.3.1 Functional Requirements

These requirements pertain to capabilities the team aspires to implement, specifically adaptive support in real-time from feedback gathered. The system will take in stakeholder text input as feedback outlining what they wish to be incorporated. The system will be able to formulate 5 to 10 context-based adaptive probing questions from stakeholder text input. The system will be able to handle multi-turn questions, wherein users can feedback answers, and in turn, receive more adaptive probing questions. The system will be able to provide more accessible conversation exports, be it in Markdown or text formats.

3.3.2 Non-Functional Requirements

Non-functional requirements are associated with quality aspects of the system from survey feedback on usability, speed, and privacy. Generated adaptive probing questions, in future evaluations, should be 85% rated useful. Therefore, questions should be relevant and clear. The system must generate responses in less than 5 seconds at 1000 words. The system must be user-friendly. The system will be able to bypass common module defaults of questions like leading questions or lack of relevance. User data will be unrecorded to ensure data privacy and support on-prem options when necessary.

3.3.3 User Requirements

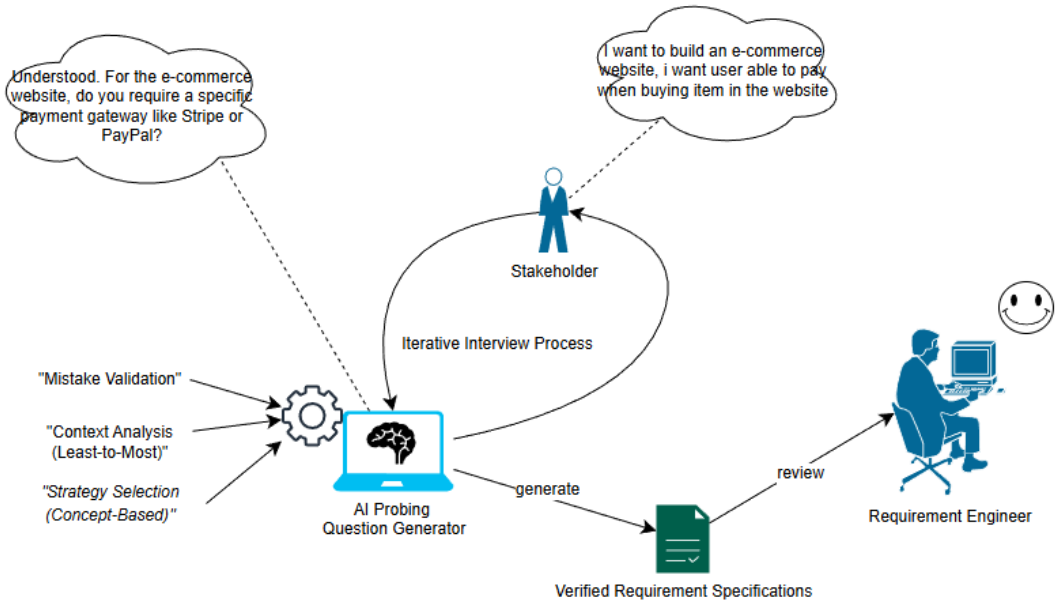
User requirements in this case will be collected in the form of user stories based on the roles and comments from the surveys. As a business analyst or requirements engineer, I would like to have an automated agent carry out preliminary interviews with the stakeholders, and I would get requirements that have been pre-validated. As a product owner/manager, I would like to have multi-turn and role differentiated questions so that I can manage stakeholder conflicting expectations. As a software developer or QA engineer, I would like to have a very basic chatbot interface that illuminates the edges so I can better participate in discussions regarding requirements without having to go through a lot of training. These requirements will be reflected in the system design and implementation guidance.

Chapter 4: System Design

4.1 Overview

In this stage of System Design, I focus on creating a more robust architectural design for the AI Probing Question Generator by detailing how I will convert functional requirements into a modular software system. The system will use the Model-View-Controller (MVC) design pattern to keep a good separation of concerns among the user interface, logic, and the data. The primary component of this design will be a Hybrid Intelligent Agent, which will enable automation of requirements elicitation and will be a result of combining 3 different research models. These include Context Management through the “Least-to-Most” prompting technique (Korn et al., 2025) for structuring interview flow, Strategy Selection with a dynamic selector (Hu et al., 2024) to vary questioning techniques, and Mistake-Driven Validation (Shen et al., 2025) for identifying questions to be presented and closing the loop to correct the invalid questions. This design will be documented using the standard Unified Modeling Language (UML) diagrams available including Section 4.2, Rich Picture Diagram, which will show automation vs. manual work, Use Case Diagram in Section 4.3 which will show actors and their interactions, Activity Diagram in Section 4.4 which will show the logic of operation, Class Diagrams and Section 4.5 and Sequence Diagrams in Section 4.6. Finally, in Section 4.7, the Interface Design will show the design of the application interface and how the user will interact with it.

4.2 Rich Picture Diagram



4.1 Rich Picture Diagram

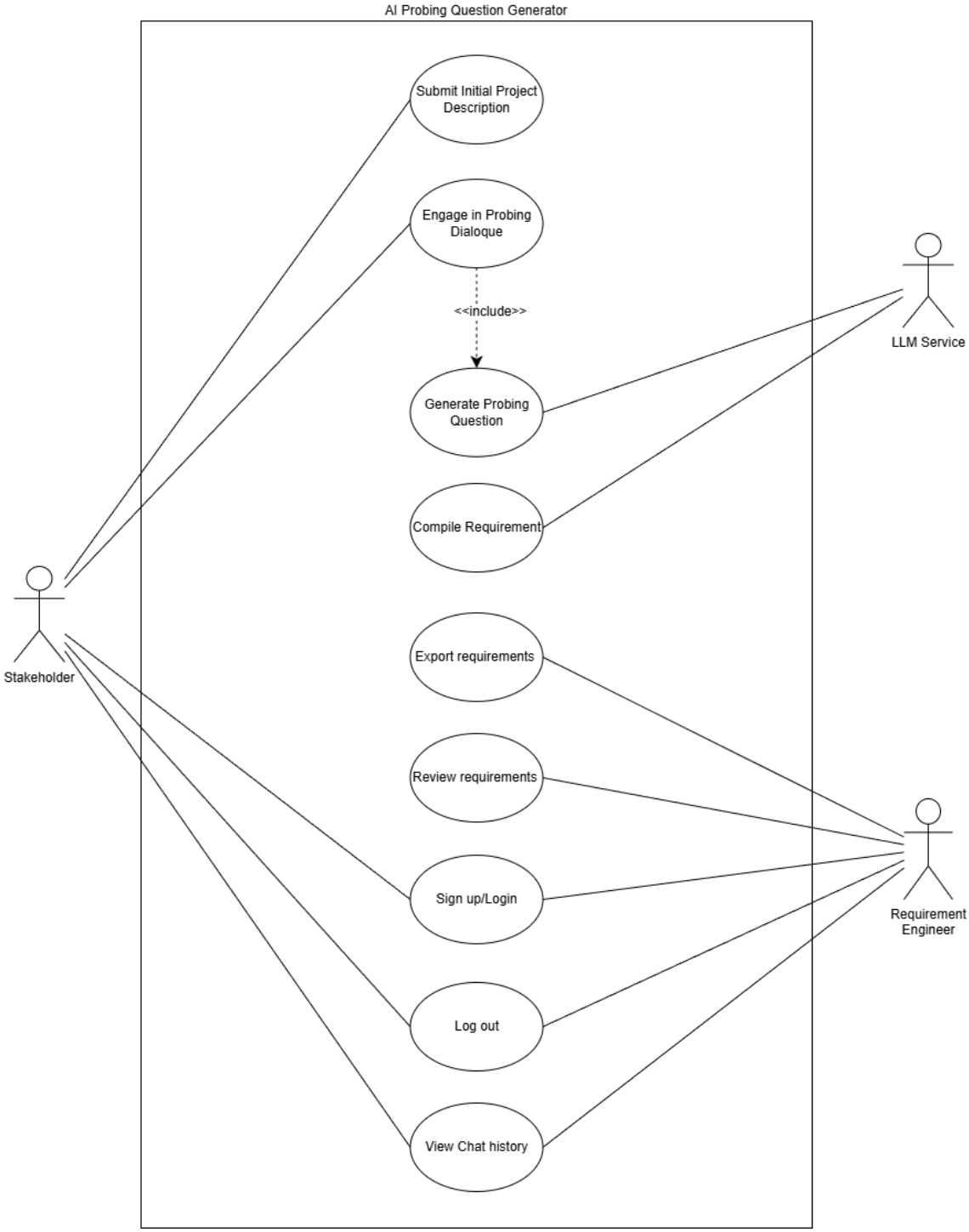
Description

Using the Rich Picture Diagram, one can see the level of abstraction provided by the AI Probing Question Generator. It looks at the automation that is proposed versus the standard ways of requirements elicitation. It identifies ways of improvements in elicitation of requirements by using automation. It looks at the center of the diagram, the “To-Be” process, where the AI subsystem plays the role of an intelligent interactor between the Stakeholder and the Requirement Engineer.

In the system being proposed, the Stakeholder is said to be participating in an Iterative Interview Process directly with the AI, which is said to be interrogating (e.g., to clarify about payment gateways) the Stakeholder from the vague descriptions given. The process of questioning is said to be governed by three types of logic, and there is an assumption that they will be contextually relevant, strategically different, and controlled in quality. The three are) Context Analysis (Least-to-Most), Strategy Selection (Concept-Based), and Mistake Validation.

The result of that conversation with the AI is supposed to be a set of Verified Requirement Specifications, and the Requirement Engineer is able to perform the final review. The design allows engineers to perform non-repetitive, and non-error prone activities, and move it to high level verification. In the estimation provided by the diagram's bottom section, the “Traditinal Way” (As-Is process) of estimation, it shows a manual repetition of work to be done between the Engineer and the Stakeholder, who is said to work with a “High Error Rate” and is “Time-Consuming” due to the High Cognitive Load of the human Interviewer.

4.3 Use Case Diagram



4.2 Use Case Diagram

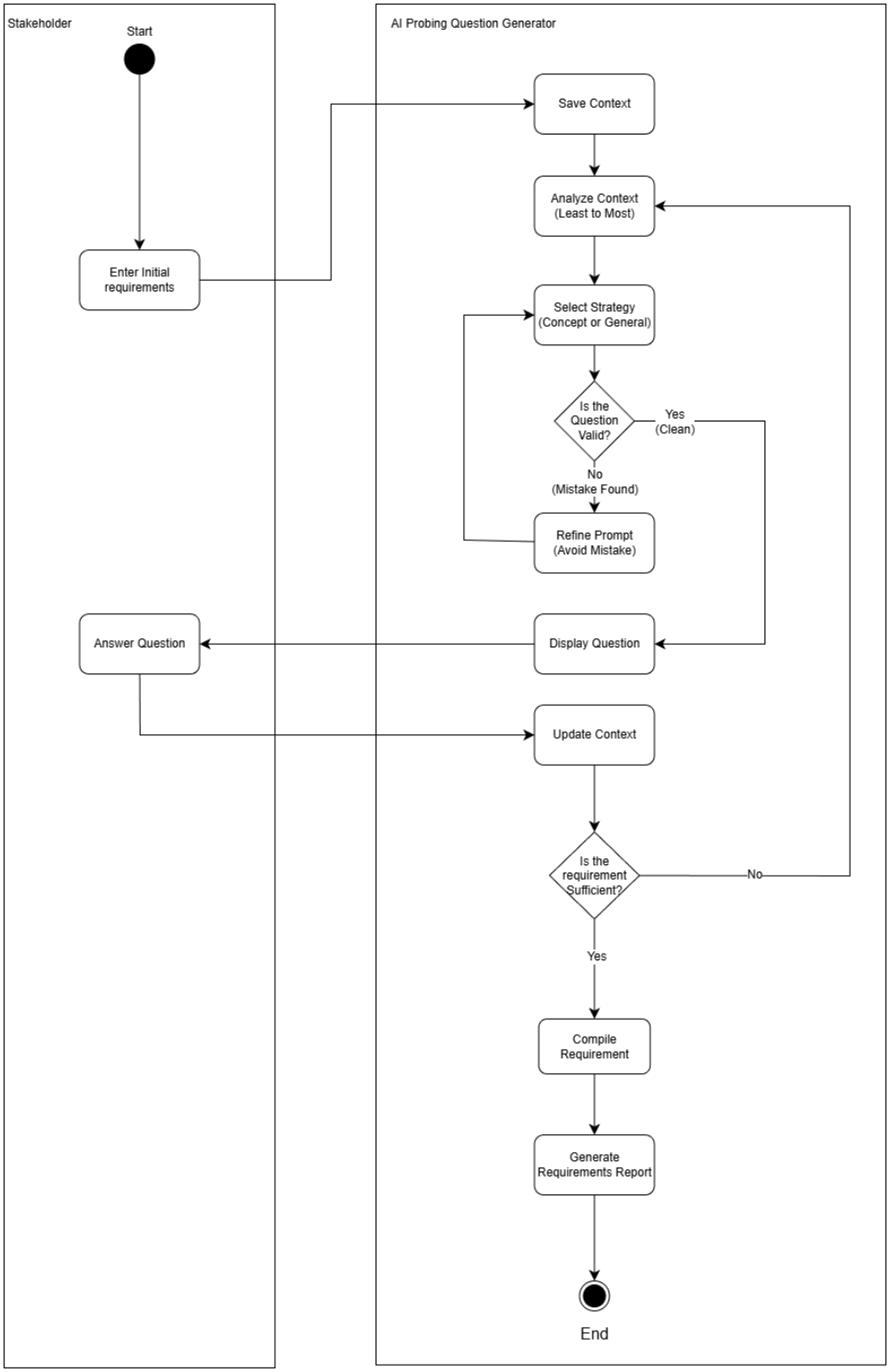
Description

The Use Case Diagram illustrates both the functional requirements and the interactions with the actors for the AI Probing Question Generator, especially focusing on the system's main human users and the external services. The system has two main actors: the Stakeholder, who outlines the primary requirements, and the Requirements Engineer, who manages the process.

The primary function is contained in the “Engage in Probing Dialogue” use case, which is triggered by the Stakeholder. The use case of "Generate Probing Question" is part of the chain, via an <> relationship, meaning that every dialogue interaction must invoke the intelligence logic on the back end. The generation process is dependent on the external LLM Service actor, showing the integration of the system with a Large Language Model for the generation and validation of questions in real time.

Subsequent to the dialogue, the system executes the “Compile Requirement” use case. This particular automated step consolidates the validated answers from the probing session and structures the data for the Requirements Engineer. The Engineer is able to exercise the “Review Requirements” and “Export Requirements” use cases to officially retrieve and evaluate the compiled data. Furthermore, the relevant utility use cases, including “Sign up/Login”, “Log out”, and “View Chat History”, are available to both actors to guarantee the secure use and continuity of the session throughout the system.

4.4 Activity Diagram



4.3 Activity diagram

Description

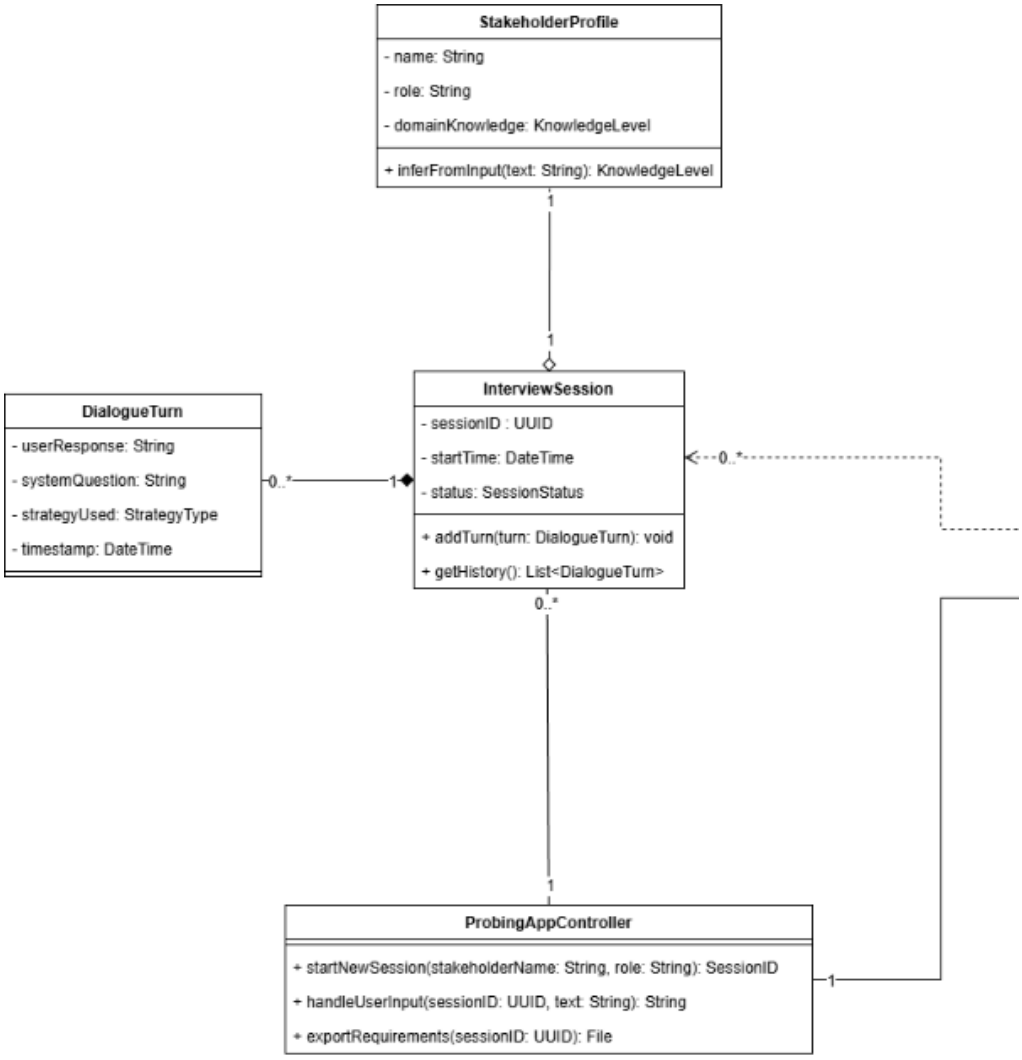
The Activity Diagram describes the operational flow of the AI Probing Question Generator and how the Stakeholder interacts with the automated System. This process is going through iterative lifecycles and designed to continuously refine Stakeholder inputs until enough detail is collected.

The process starts with the Stakeholder providing a basic description of the project. The system then does a Save Context and starts the session. Next, the system's Analyze Context activity uses the Least-to-Most prompting technique to determine the question format based on the interview phase. The flow then proceeds to Select Strategy, where the system evaluates the best questioning type (Concept-Based, General, etc.) to ensure there is no repetition in the dialogue.

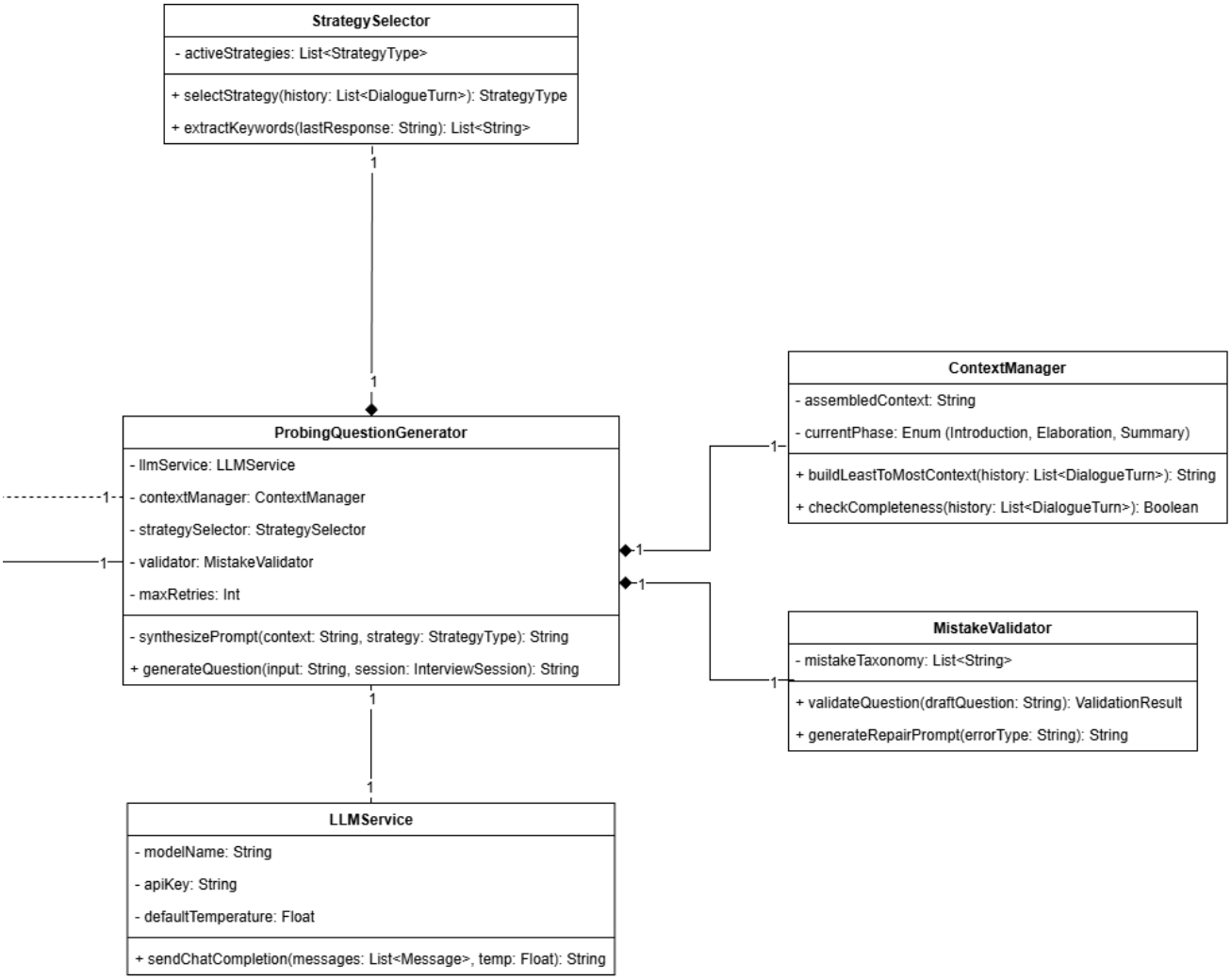
An important control point in the flow is the decision point Is the Question Valid?. This point applies a mistake-driven validation approach. If the question is considered invalid (for example, the question is wrong), the flow diverges to Refine Prompt so the system can self-correct and modify the question to the better version of the question.

Once a legitimate question is generated, it is Showcased to the Stakeholder. The Stakeholder's reply is recorded during the 'Answer Question' activity and is used to Update the Context. Following this, the system inspects the 'Is the requirement Sufficient?' decision node. Should the system evaluate the newly acquired data as insufficient, the flow returns to the context and analysis phase to investigate further. If, however, the information provided is adequate, the system will break from the iterative cycle to Commence the Requirements and ultimately Produce the Requirements Report, concluding the process.

4.5 Class Diagram



4.4.1 Left Part of Class Diagram (continue)



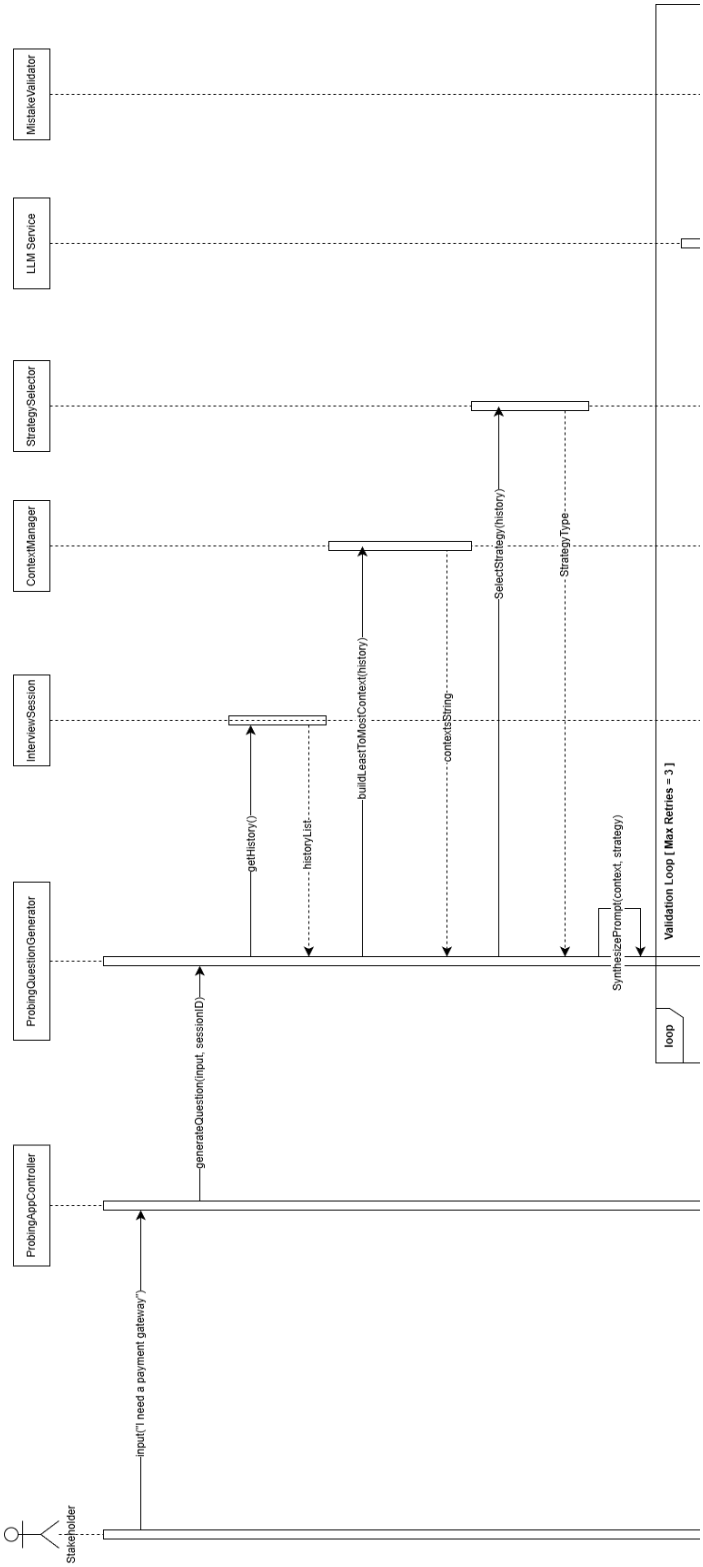
4.4.2 Right Part of Class Diagram

Description

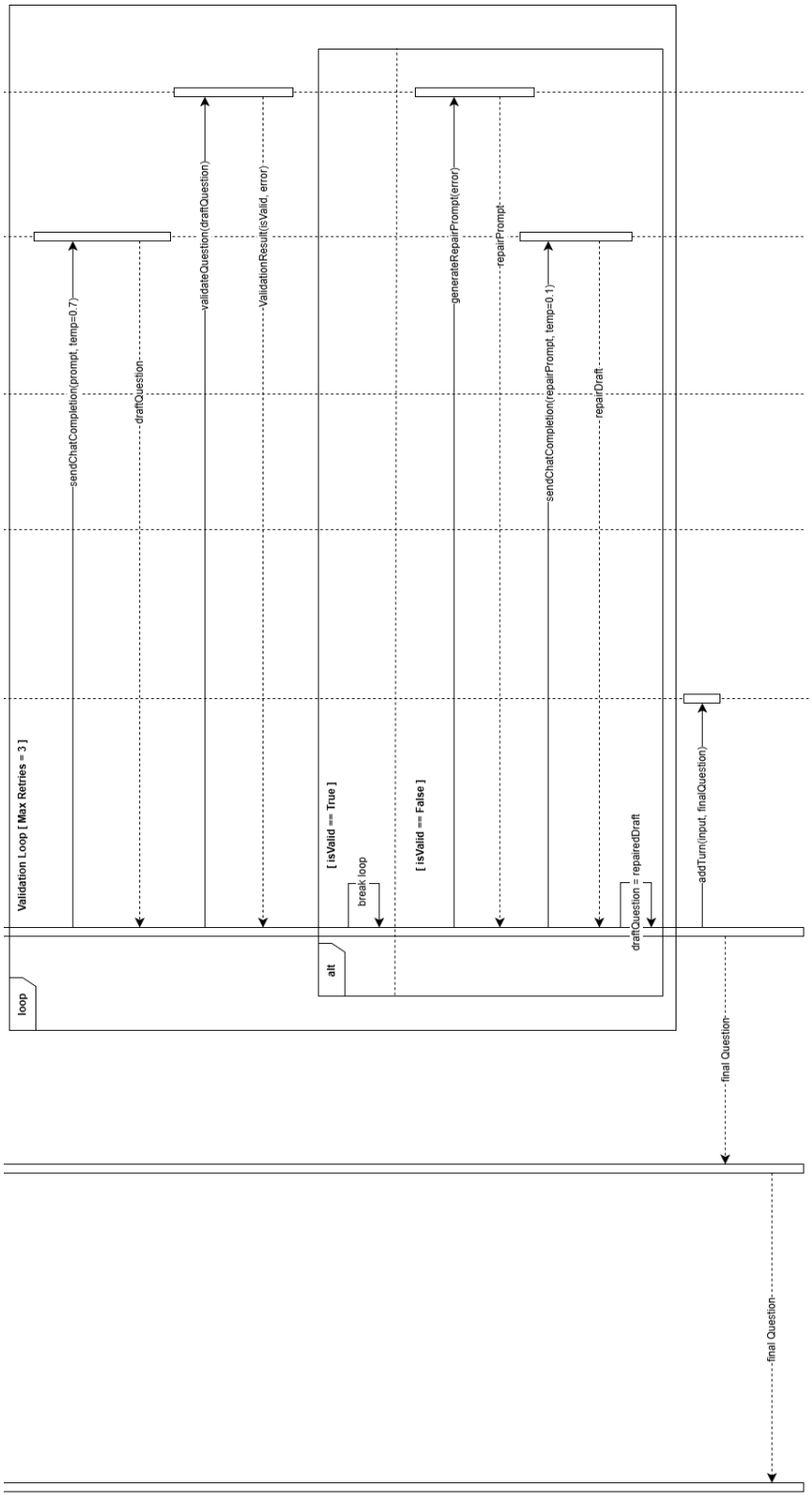
The Class Diagram presents the static structure of the AI Probing Question Generator, detailing the system's core components, attributes, and relationships. The architecture follows a Hybrid Intelligent Agent design, where the central ProbingQuestionGenerator class acts as the primary orchestrator. This class does not function in isolation but relies on strict Composition relationships (denoted by filled diamonds) with three specialized logic components: the ContextManager, which structures the interview flow using the 'Least-to-Most' technique; the StrategySelector, which determines the optimal questioning strategy to ensure variety; and the MistakeValidator, which implements a quality control mechanism to detect and correct errors in generated questions.

The system's interaction with the external AI model is managed by the LLMService class, which handles API authentication and dynamic parameter tuning (e.g., temperature settings) via a direct association. The ProbingAppController serves as the entry point for the application, managing the lifecycle of each InterviewSession. Each session maintains a one-to-many relationship with DialogueTurn objects to track conversation history and aggregates a StakeholderProfile, which dynamically infers the user's domain knowledge level to adjust the system's output complexity.

4.6 Sequence Diagram



4.5.1 Sequence Diagram (Continue)



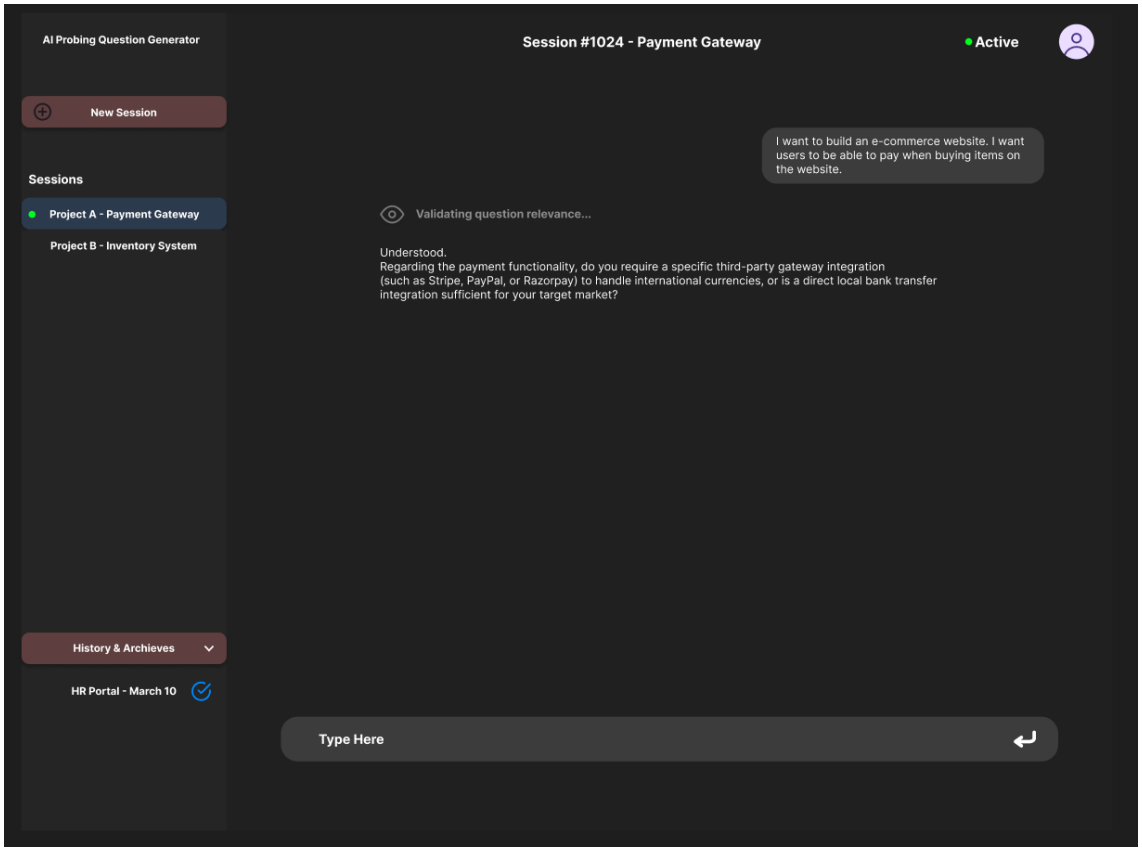
4.5.2 Sequence Diagram

Description

The Sequence Diagram describes the runtime relationships for the use case “Generate Probing Question” and how the system objects pass messages to one another. The first part of the process begins with the ProbingAppController, who takes a requirement description from the Stakeholder and calls the ProbingQuestionGenerator. The generator, first, obtains the conversation history from the InterviewSession and sends it to the ContextManager to formulate a contextually relevant prompt. At the same time, the StrategySelector analyzes the history and selects the best strategy to use for questioning.

A noteworthy component of this sequence is the Validation Loop (within the loop fragment). The system first drafts a question with high temperature (0.7) to increase diversity. This draft is first sent to the MistakeValidator. If the system detects a leading question or an ambiguous question, it goes into self-correction mode. This means the draft is sent along with a specific correcting prompt to the LLMService, and the temperature is set to low (0.1) to enforce the correction. This cycle of validation is completed three times (defined by Max Retries) so the system does not become too unreliable. When an appropriate question is detected, it is saved to the session (via addTurn) and sent back to the Stakeholder.

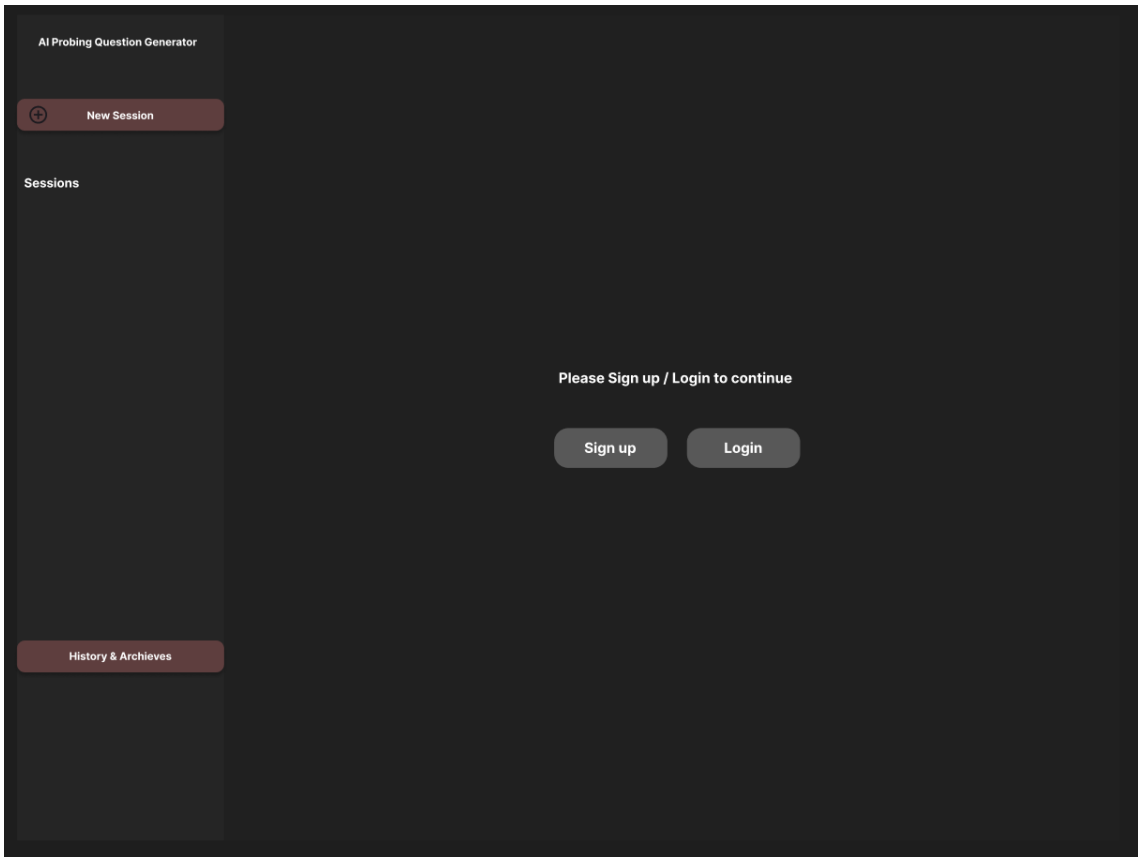
4.7 Interface Design



4.6 Figure of Interface Design

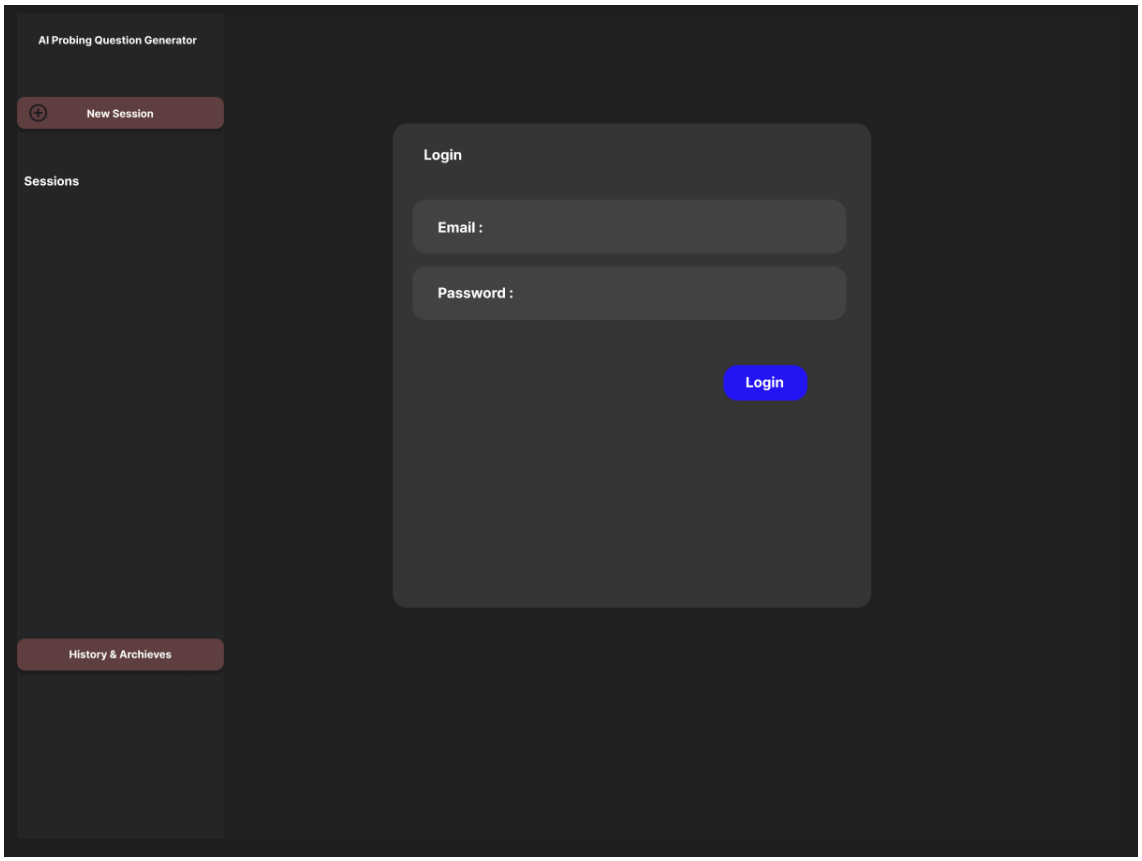
Upon entering the application, the first thing that will greet you is the Main Screen. To the left is the Session Management Console, which gives you the means to control your workspace. You will see the 'New Session' button which allows the Requirements Engineer to start a new project. Below this is a list of ongoing 'Sessions' which records the progress of all your projects. For instance, you might spot 'Project A. Payment Gateway' plus a green dot signifying that there is an active Interview Session running in the background. The 'History & Archives' section is for retrieving records like the 'HR Portal' in order to help control the clutter in the active workspaces.

The Main Dialogue Area (Center Panel) is the most important part of the application. The header shows the Session ID and Status (“#1024 - Active”), showcasing the system’s status management. The Interface’s most important feature is the System Status Indicator just above the AI’s response area. The text “Validating question relevance…” explains what the system is doing, and provides the user with a visual indication that the MistakeValidator component is working on the draft question before it is displayed. This feature shows the stakeholder that the system is doing a quality assurance check. The system clearly separates the Stakeholder’s text (Right-aligned) and the AI’s probing questions (Left-aligned) and there is a text input field at the bottom of the screen. Main Dialogue Area is the biggest area of the application and has most of the information. At the top of the screen, it shows the Session ID and the system status, Active or Inactive. Idr is the system is working well.



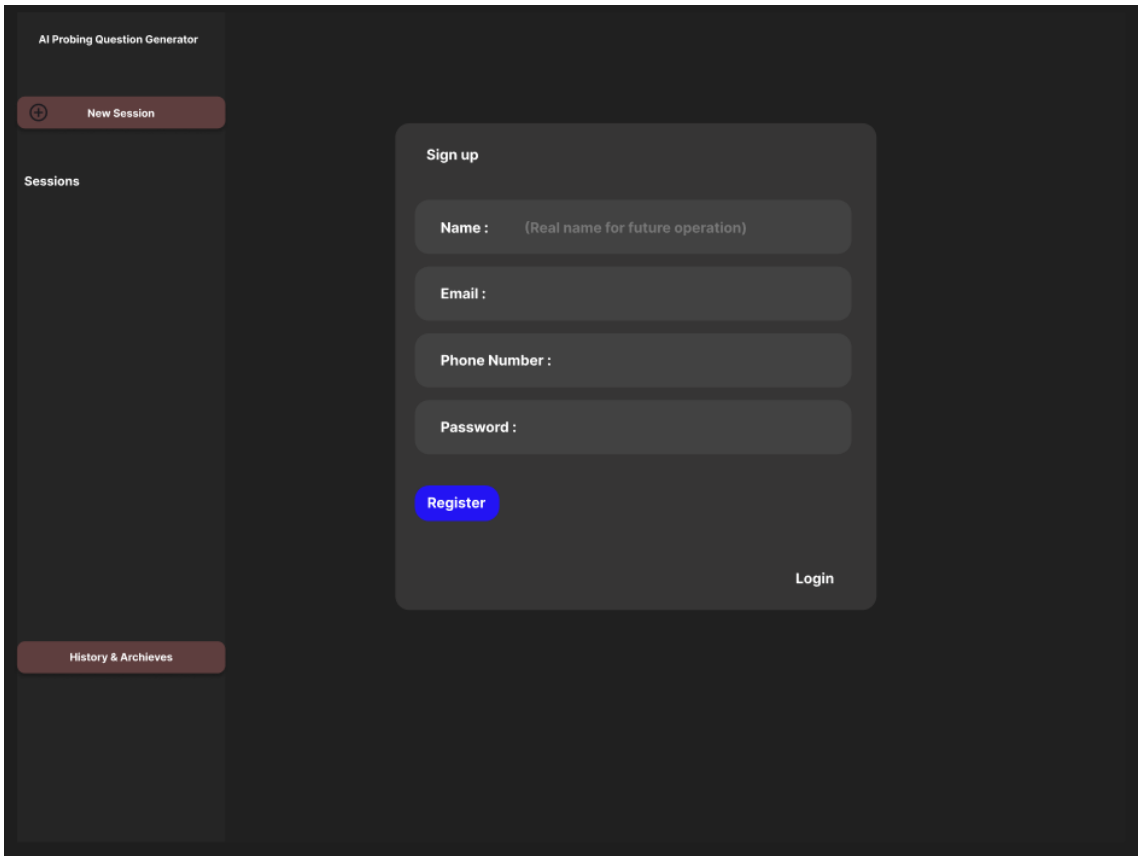
4.7 Figure of Signup/Login Interface

When users access the site interface for the first time, they are greeted by the option to either log in or sign up. This view serves as a first line system gatekeeper. All users who access the app and are not authenticated are shown a simple message: "Please sign up / log in to continue." In this state, the system also prevents the sidebar from displaying the "New Session" option, and as such, no elicitation data can be generated or retrieved without a logged-in user.



4.8 Figure of Login Interface

The Login Interface prompts existing users for their credentials (Email and Password). The design features a high-contrast "Login" action button to guide the user. This interface is the frontend entry point for the ProbingAppController to verify user credentials against the database and retrieve the associated StakeholderProfile



4.9 Figure of Signup interface

The Sign-Up Interface captures user meta-data needed to fulfill the StakeholderProfile class. In addition to standard user credentials (Email, Password), the form requests the user's Real Name (the form has the placeholder text “Real name for future operation”). This draws significance as it helps ensure that all generated requirement reports are tied to a specific human stakeholder so that accountability isn’t lost in the requirements engineering process. A field for “Phone Number” is included for multi-factor authentication or for contact-coping. Users may toggle back to the “Login” screen via the link at the bottom right of the modal, creating a seamless experience.

4.8 Summary

In Chapter 4, we outline the modular system design for the AI Probing Question Generator based on the system requirements from Chapter 3. This chapter explains the choice of the Model-View-Controller (MVC) design pattern for the system's logic decentralization, and a Hybrid Intelligent Agent for logic, which incorporates the latest research in context, strategy, and mistake-based validation.

The design is expressed in multiple inter-related models. The Rich Picture Diagram offered a conceptual illustration of the automation system's workflow, and compared the efficiency of the proposed design to the elicitation process. The Use Case and Activity Diagrams described the processes and operational logic, including the decision logic for valid question generation. The Class and Sequence Diagrams described how the components of the ProbingQuestionGenerator implement the "Validation Loop" to help improve reliability. Lastly, the Interface Design illustrated how the user dashboard, is designed to mask the complex backend processes.

These design artifacts collectively ensure the requirements of the project are met and the system is ready for the implementation phase, which is described in the following chapters.

Chapter 5: Implementation Plan

5.1 Overview

This chapter describes the particular order of steps that will be taken for the building, testing, and execution of the AI Probing Question Generator for the next semester (Project II). The steps will be done in order of the Waterfall methodology as explained in Chapter 1, and will ensure that all the verified steps are completed before moving on to the next step.

5.2 Development Phase

The Development Phase will create a working software solution from the system design specifications in Chapter 4. The process will be broken down into three stages to ensure the system is modular and maintainable.

Tools might be used:

Table 5.1 Tools Used for Development Phase

Category	**Tool/Technology**	**Justification**
Frontend	React.js / HTML5 / CSS	Selected for responsive design and component-based UI rendering (as seen in Section 4.7).
Backend	FastAPI	Chosen for robust API handling and ease of integration with LLM libraries.
Database	MongoDB / Firebase	Used to store database

Category	Tool/Technology	Justification
AI/LLM	Gemini API	The core intelligence engine used for text generation and validation.
Version Control	Git / GitHub	Used for source code management and version tracking.

5.2.1 Frontend Development

Once the backend is stable and functioning, the Frontend will be built according to the High-Fidelity Figma designs in Section 4.7. Development will focus on the construction of the user interfaces for the “Session Management Console” (Left Sidebar) to allow for navigation of active and archived interviews, as well as the main dialogue area (e.g., “Validating Question…”) where system status feedback is given in real-time and the agent is processing. Also, the “Live Requirements” panel (Right Sidebar) will be constructed to be dynamic and automatically update to ensure that the stakeholder has real-time visibility into the elicitation process once the backend has compiled enough requirements.

5.2.2 Backend Development

The first phase will build the server-side components. Web frameworks like Python FastAPI will be used to process API calls. Considerable attention will be directed to the development of the Hybrid Intelligent Agent. This involves the development of the three core intelligence modules designed in the Class Diagram, including: the Context Manager to monitor interview stages using “Least-to-Most” prompting, the Strategy Selector to adaptively control questioning strategies, and the Mistake Validator to identify and/or modify questions considered invalid. At the same time, secure integration with the Large Language Model (LLM) will be established, as well as the tuning of some parameters, including temperature, to stabilize the generation between imaginative and factual.

5.2.3 Database Implementation

At the same time, the relational database will be designed using the database schema to obtain the persistent storage of the data. This includes the design of the StakeholderProfile tables, which will store metadata of the user and level of domain knowledge. Concurrently, the system will prepare the structures of the InterviewSession as well as of the DialogueTurn to log the history of the conversation and facilitate the context retention in the multiple-turn interactions. This structure is important for the proper functioning of the Context Manager. It enables the AI to refer to the previous answer when generating new probing questions.

5.3 Testing Phase

As a matter of principle, with reliance being one of the most crucial aspects of evaluating the system, and accuracy being the most crucial after development, an entire strategy will be created after Development is finished, and the Development testing strategy will span the entire system.

5.3.1 Unit Testing

One of the most supportive testing methods is testing a single unit, or a group of interrelated units, to see if the system is functionally and logically correct. The greatest scope is the system's Strategy Selector, to determine if the system will produce the correct and logically valid type of strategy for the event of a conversation. The greatest scope apart from the previous one is the Context Manger, to determine if the system will produce the correct and logically valid type of strategy for an event when the system is meant to or is to never be prevented from being stuck in the loop. The greatest scope is repeated.

5.3.2 Integration Testing

This phase will confirm the functioning system between different modules, most importantly the flow of data between the core logic and the ancillary services. The Scope will validate that the ProbingQuestionGenerator sends instructions to the LLMService and retrieves answers without encountering a timeout error. The crucial part of testing will be to validate the functioning of the quality control loop by testing if the Mistake Validator retrieves and fixes "bad" drafts (e.g., leading questions) before they are sent to the controller.

5.3.3 System Testing

In this phase, we will perform end-to-end testing for the system as a whole and compare it with the functional requirements listed in Chapter 3. This will mean verifying the entire "Engage in Probing Dialogue" use case, i.e. the logging in, starting a new session, question generation, question validation, and the exporting of the requirements report. This phase will also ensure that all separate components of the system are integrated to provide the intended user experience.

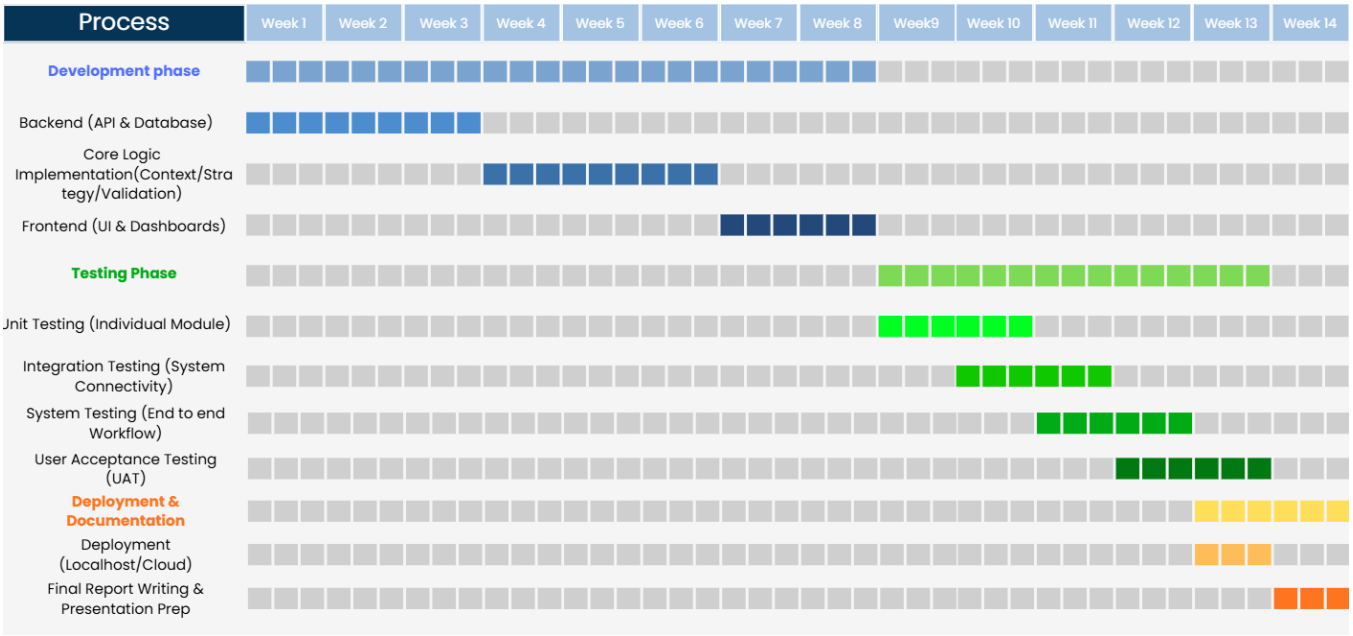
5.3.4 User Acceptance Testing

The last testing phase will involve a human test to evaluate the system's utility in practical situations. A group of test subjects, consisting of some Stakeholders and a few Requirements Engineers, will interact with the system to role-play complete elicitation sessions. User feedback will be obtained on the quality of the probing questions, the precision of the error detection, as well as the overall experience with the user interface. This feedback will constitute the main input for the last round of development on the system.

5.4 Development Phase

The last part of the project will center on the deployment and presentation of the system. The application will be deployed to a local hosting environment first to achieve optimal performance and zero latency for the final viva presentation. Time permitting, a secondary deployment for the application will be conducted on a cloud hosting environment to show the examiners the system's accessibilty and web-based features. This will be the last phase of the project with the preparation of the final documentation and presentation of the functional AI Probing Question Generator to show that the system meets the definded functional and non-functional requirements of the project.

5.5 Gantt Chart



5.1 Figure of Gantt Chart

Chapter 6 : Conclusion

6.1 Description

The project's main goal was to provide a solution to a software engineering problem. This problem pertains to the inefficiencies and inaccuracies of manually executed elicitation of requirements. This is especially the case when stakeholder input is vague and cognitive load is higher for the analysts. As such, the AI Probing Question Generator was introduced. This is an intelligent web-based assistant that is able to autonomously resolve ambiguities in requirements by means of context-based probing.

The project's main goal was to provide a solution the first phase of the project, hereafter referred to as Project I, the initial groundwork has been methodically done. For example, the comprehensive literature review in chapter two brought the strengths and shortcomings of existing techniques for prompting to the fore. This review was also used to formulate a vision for the “Hybrid Intelligent Agent”. This encapsulates the Least-to-Most Prompting for context management, Strategy Selection for diversity, and Mistake-Driven Validation for the quality control on to a software engineering problem. This problem pertains to the inefficiencies and inaccuracies of manually executed elicitation of requirements. This is especially the case when stakeholder input is vague and cognitive load is higher for the analysts. As such, the AI Probing Question Generator was introduced. This is an intelligent web-based assistant that is able to autonomously resolve ambiguities in requirements by means of context-based probing.

The System Design in chapter four was the last in a line of these requirements in real-time prompting for support. This designed a system to meet the requirements in an architectural framework by means of a Model-View-Controller (MVC) pattern and visualized the essence of the system using Unified Modeling Language (UML) to rationally structured diagrams and marked its interfaces substantially. It is safe to state that the entire project is now ready to be executed.

References

1. Shen, Y., Singhal, A., & Breaux, T. (2025). Requirements elicitation follow-up question generation. arXiv:2507.02858.
   Requirement elicitation followup question generation

2. Hu, J., Guo, J., Tang, N., Ma, X., Yao, Y., Yang, C., & Xu, Y. (2024). Designing the conversational agent: Asking follow-up questions for information elicitation. Proceedings of the ACM on Human-Computer Interaction, 8(CSCW1), Article 43. Design follow up questions agent

3. Korn, A., Gorsch, S., & Vogelsang, A. (2025). LLMREI: Automating Requirements Elicitation Interviews with LLMs. arXiv preprint arXiv:2507.02564. https://arxiv.org/pdf/2507.02564

Appendix A

Meeting Log 1

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester OCT / NOV 2025 (Trimester ID:2530)

Meeting Date: **4/11/2025**	**Meeting No.:** **1**
Meeting Mode: Online	
Project ID: **683**	**Project Type:** Application-based
Project Title : **Automatic Generation of Probing Questions**	
Student ID : **243UC247NS**	**Student Name:** **Yam Kar Yong**
Student Programme and Specialisation: **Computer Science in Software Engineering**	
Supervisor Name: **Dr. Lim Tek Yong**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE
[Please write the details of the work done, after the last meeting]

Tasks: Problem Formulation and Project Planning / Background Study or Literature Review / Requirement Analysis or Theoretical Framework / Design or Research Methodology / Prototype Development or Proof of Concept / Draft Report Completion

(Please strike out the tasks, which are not applicable)

Details (in point form):

2. WORK TO BE DONE
[Please write the details of the work to be done, before the next meeting]

Tasks: Problem Formulation and Project Planning

(Please strike out the tasks, which are not applicable)

Details (in point form):

1. Abstract
2. Introduction
3. Problem statement
4. Objective
5. Scope
6. Deliverable

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

No problem encountered

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

tekyong

.....
Supervisor's Signature

.....
Student's Signature

.....

.....

Meeting Log 2

MMU MULTIMEDIA UNIVERSITY | FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester OCT / NOV 2025 (Trimester ID:2530)

Meeting Date: **18/11/2025**	**Meeting No.:** **2**
Meeting Mode: Face-to-Face	
Project ID: **683**	**Project Type:** Application-based
Project Title : **Automatic Generation of Probing Questions**	
Student ID : **243UC247NS**	**Student Name:** **Yam Kar Yong**
Student Programme and Specialisation: **Computer Science in Software Engineering**	
Supervisor Name: **Dr. Lim Tek Yong**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE
[Please write the details of the work done, after the last meeting]

Tasks: Problem Formulation and Project Planning

(Please strike out the tasks, which are not applicable)

Details (in point form):

1. Abstract
2. Introduction
3. Problem statement
4. Objective
5. Scope
6. Deliverable

2. WORK TO BE DONE
[Please write the details of the work to be done, before the next meeting]

Tasks: Problem Formulation and Project Planning / Background Study or Literature Review

(Please strike out the tasks, which are not applicable)

Details (in point form):

1. Change paragraph to justify
2. Correct Problem statement, Objective, Project Scope
3. Do the literature review for Chapter 2

3. PROBLEMS ENCOUNTERED AND SOLUTIONS
[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

No problem encountered

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

tekyong

.....

Supervisor's Signature

.....

Student's Signature

.....

.....

Meeting Log 3

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester OCT / NOV 2025 (Trimester ID:2530)

Meeting Date: **2/12/2025**	**Meeting No.:** 3
Meeting Mode: Face to Face	
Project ID: **683**	**Project Type:** Application-based
Project Title : **Automatic Generation of Probing Questions**	
Student ID : **243UC247NS**	**Student Name:** **Yam Kar Yong**
Student Programme and Specialisation: **Computer Science in Software Engineering**	
Supervisor Name: **Dr. Lim Tek Yong**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE
[Please write the details of the work done, after the last meeting]

Tasks: Background Study or Literature Review

(Please strike out the tasks, which are not applicable)

Details (in point form):

- Correct Chapter 1
- Done the literature review for Chapter 2

2. WORK TO BE DONE
[Please write the details of the work to be done, before the next meeting]

Tasks: Background Study or Literature Review

(Please strike out the tasks, which are not applicable)

Details (in point form):

- Modify literature review (research deeply)
- Change paper to review
- Do survey

3. PROBLEMS ENCOUNTERED AND SOLUTIONS
[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

No problem encountered yet

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

tekyong
.....
Supervisor's Signature

.....
Student's Signature

.....
Co-Supervisor's Signature
(If applicable)

.....
Company Supervisor's Signature
(If applicable)

Meeting Log 4

MULTIMEDIA UNIVERSITY | FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester OCT / NOV 2025 (Trimester ID:2530)

Meeting Date: **16/12/2025**	**Meeting No.:** **4**
Meeting Mode: Face to Face	
Project ID: **683**	**Project Type:** Application-based
Project Title : **Automatic Generation of Probing Questions**	
Student ID : **243UC247NS**	**Student Name:** **Yam Kar Yong**
Student Programme and Specialisation: **Computer Science in Software Engineering**	
Supervisor Name: **Dr. Lim Tek Yong**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE
[Please write the details of the work done, after the last meeting]

Tasks: Background Study or Literature Review

(Please strike out the tasks, which are not applicable)

Details (in point form):

- Changed Paper to review
- Modified Literature review
- Do changes on Chapter 1
- Done the survey form

2. WORK TO BE DONE
[Please write the details of the work to be done, before the next meeting]

Tasks: Literature Review and Post survey form

(Please strike out the tasks, which are not applicable)

Details (in point form):

- Post survey form to social media
- Do changes on literature review

3. PROBLEMS ENCOUNTERED AND SOLUTIONS
[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

No problem encountered yet

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

tekyong

Supervisor's Signature

Student's Signature

Meeting Log 5

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester OCT / NOV 2025 (Trimester ID:2530)

Meeting Date: **23/12/2025**	**Meeting No.:** 5
Meeting Mode: Face to Face	
Project ID: **683**	**Project Type:** Application-based
Project Title : **Automatic Generation of Probing Questions**	
Student ID : **243UC247NS**	**Student Name:** **Yam Kar Yong**
Student Programme and Specialisation: **Computer Science in Software Engineering**	
Supervisor Name: **Dr. Lim Tek Yong**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE
[Please write the details of the work done, after the last meeting]

Tasks: Background Study, Literature Review and post survey

(Please strike out the tasks, which are not applicable)

Details (in point form):

- Posted survey through social media
- Modified Literature review
- Do changes on Chapter 1

2. WORK TO BE DONE
[Please write the details of the work to be done, before the next meeting]

Tasks: Literature Review and Post survey form

(Please strike out the tasks, which are not applicable)

Details (in point form):

- Get result from survey
- Do changes on literature review
- Modify some part on chapter 1

3. PROBLEMS ENCOUNTERED AND SOLUTIONS
[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

No problem encountered yet

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

tekyong
.....
Supervisor's Signature

.....
Student's Signature

.....
Co-Supervisor's Signature
(If applicable)

.....
Company Supervisor's Signature
(If applicable)

Meeting Log 6

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester OCT / NOV 2025 (Trimester ID:2530)

Meeting Date: **30/12/2025**	**Meeting No.:** **6**
Meeting Mode: Face to Face	
Project ID: **683**	**Project Type:** Application-based
Project Title : **Automatic Generation of Probing Questions**	
Student ID : **243UC247NS**	**Student Name:** **Yam Kar Yong**
Student Programme and Specialisation: **Computer Science in Software Engineering**	
Supervisor Name: **Dr. Lim Tek Yong**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE
[Please write the details of the work done, after the last meeting]

Tasks: Background Study, Literature Review and post survey

(Please strike out the tasks, which are not applicable)

Details (in point form):

- Get result from survey for 30+ person
- Modified literature review

2. WORK TO BE DONE
[Please write the details of the work to be done, before the next meeting]

Tasks: Literature Review and Post survey form

(Please strike out the tasks, which are not applicable)

Details (in point form):

- Write Chapter 3 for each result section collected
- Do chapter 4 (system design), Diagrams
- Modify some error from chapter 1, chapter 2

3. PROBLEMS ENCOUNTERED AND SOLUTIONS
[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

No problem encountered yet

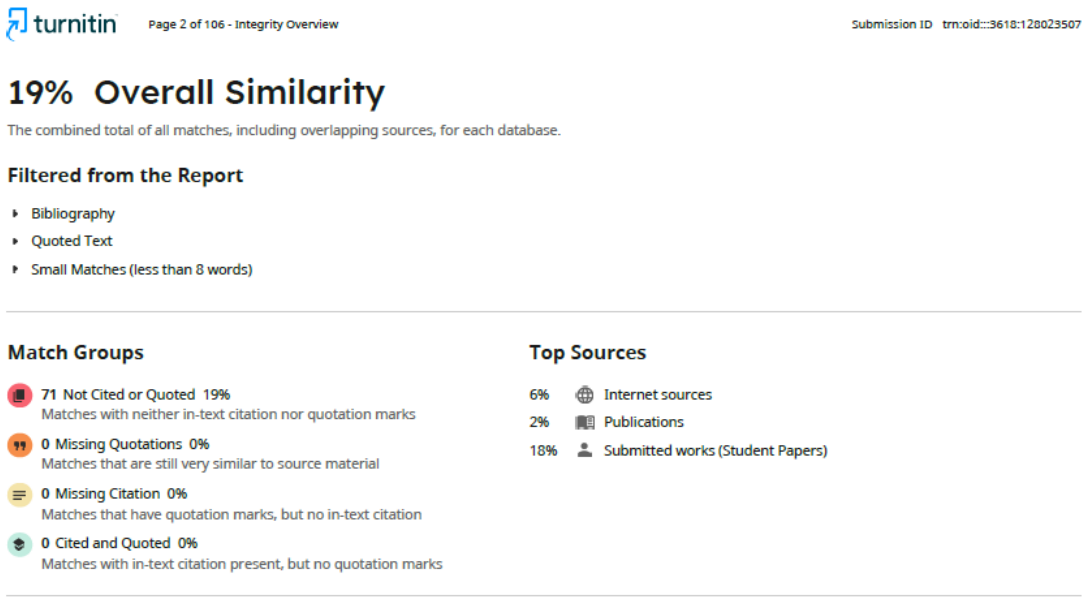
4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

Supervisor's Signature

Student's Signature

Appendix B

Turnitin similar report



turnitin Page 2 of 106 - Integrity Overview Submission ID trn:oid:::3618:128023507

19% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

- 71 Not Cited or Quoted 19%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 2% Publications
- 18% Submitted works (Student Papers)

turnitin Page 3 of 106 - Integrity Overview

Match Groups

- 71 Not Cited or Quoted 19%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
2% Publications
18% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	
Multimedia University on 2026-02-11		8%
2	Student papers	
Multimedia University on 2026-02-09		2%
3	Student papers	
Multimedia University on 2025-02-09		<1%
4	Student papers	
Multimedia University on 2025-02-09		<1%
5	Student papers	
Multimedia University on 2026-02-09		<1%
6	Student papers	
Multimedia University on 2026-02-09		<1%
7	Internet	
www.doria.fi		<1%
8	Student papers	
University of Limerick on 2016-09-06		<1%
9	Student papers	
Multimedia University on 2021-11-17		<1%
10	Student papers	
Multimedia University on 2025-02-04		<1%